Part I - Ford GoBike System Data Exploration

by Khoa Pham

Table of Contents

- Introduction
- Preliminary Wrangling
- Cleaning Data
- Enriching features
- Univariate Exploration
- Bivariate Exploration
- Multivariate Exploration
- Conclusion

Introduction

> This data set includes information about individual rides made in a bike-sharing system covering the greater San Francisco Bay area in 2019. Bay Wheels is a regional public bicycle sharing system in California's San Francisco Bay Area. It is operated by Motivate in a partnership with the Metropolitan Transportation Commission and the Bay Area Air Quality Management District. They provides 12 seperated csv files for every month in 2019.

Preliminary Wrangling

In [ ]:
```
# import all packages and set plots to be embedded inline
import numpy as np
from numpy import median
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
import glob

import warnings
warnings.filterwarnings("ignore")

# allows us to view our graphs in jupyter notebook.
%matplotlib inline
```

There are 12 csv files of Baywheels trip data acordingly for 12 months of 2019 in San Francisco Bay area. As a result, the following block of code is for looping to the path and union all the csv file into one call "2019_Baywheels-tripdata.csv". We are going to use this file for analysis onwards. Because of using local machine for better performance of processing. I convert these line of code to comment to the reduce the time of process.

In [ ]:
```
#There are 12 csv files for the Baywheels trip data for 12 months in San Franci
#for f in glob.glob('*/2019_Baywheels/*.csv'):
#    print(f.split("/")[-1])
```

In [ ]:
```
# Unite all the csv to a single dataframe
#df_csv_union = pd.concat([pd.read_csv(f, low_memory=False) for f in glob.glob
```

In [ ]:
```
#Write to csv for usage
#df_csv_union.to_csv("2019_Bayweels-tripdata.csv")
```

In [ ]:
```
# Start read the united csv files
df = pd.read_csv('2019_Baywheels-tripdata.csv', low_memory=False)
```

In [ ]:
```
df.sample(5)
```

Out[ ]:

	Unnamed: 0	duration_sec	start_time	end_time	start_station_id	start_station_
980733	980733	1002	2019-05-12 16:24:30.9940	2019-05-12 16:41:13.7460	110.0	17th & Folsom Park (17t Fols
491832	491832	518	2019-03-18 16:57:14.9780	2019-03-18 17:05:53.6300	66.0	3rd St at Tow
2446702	2446702	262	2019-12-07 19:03:04.9880	2019-12-07 19:07:27.5210	87.0	Folsom St at
2022178	2022178	380	2019-10-20 14:21:27.4100	2019-10-20 14:27:47.6230	240.0	Hast Telegra
1977268	1977268	800	2019-10-25 16:03:25.1440	2019-10-25 16:16:45.8850	43.0	San Francisco Library (Grov

In [ ]:
```
#Drop unwanted column "Unamed: 0" that is the duplicated index column.
df.drop(columns = df.columns[0], axis=1, inplace=True)
```

In [ ]:
```
# Recheck the columns
df.head()
```

Out[ ]:

	duration_sec	start_time	end_time	start_station_id	start_station_name	start_static
0	80825	2019-01-31 17:57:44.6130	2019-02-01 16:24:49.8640	229.0	Foothill Blvd at 42nd Ave	
1	65900	2019-01-31 20:58:33.8860	2019-02-01 15:16:54.1730	4.0	Cyril Magnin St at Ellis St	
2	62633	2019-01-31 18:06:52.9240	2019-02-01 11:30:46.5300	245.0	Downtown Berkeley BART	
3	44680	2019-01-31 19:46:09.7190	2019-02-01 08:10:50.3180	85.0	Church St at Duboce Ave	
4	60709	2019-01-31 14:19:01.5410	2019-02-01 07:10:51.0650	16.0	Steuart St at Market St	

In [ ]:
```
# Check how many columns, rows of the dataset are.
print(f" There are {df.shape[0]} rows in the dataset.")
print(f" There are {df.shape[1]} columns in the dataset.")
```

```
 There are 2506983 rows in the dataset.
 There are 15 columns in the dataset.
```

In [ ]:
```
# Data type checking
df.info(null_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2506983 entries, 0 to 2506982
Data columns (total 15 columns):
 #   Column                   Non-Null Count    Dtype  
---  ------                   --------------    -----  
 0   duration_sec             2506983 non-null  int64  
 1   start_time               2506983 non-null  object 
 2   end_time                 2506983 non-null  object 
 3   start_station_id         2426249 non-null  float64
 4   start_station_name       2426850 non-null  object 
 5   start_station_latitude   2506983 non-null  float64
 6   start_station_longitude  2506983 non-null  float64
 7   end_station_id           2424081 non-null  float64
 8   end_station_name         2424633 non-null  object 
 9   end_station_latitude     2506983 non-null  float64
 10  end_station_longitude    2506983 non-null  float64
 11  bike_id                  2506983 non-null  int64  
 12  user_type                2506983 non-null  object 
 13  bike_share_for_all_trip  2263724 non-null  object 
 14  rental_access_method     120838 non-null   object 
dtypes: float64(6), int64(2), object(7)
memory usage: 286.9+ MB
```

In [ ]:
```
#Check duplicated value of dataset
print(f"The total number of duplicated value of the dataset: {sum(df.duplicate
```

```
The total number of duplicated value of the dataset: 0
```

In [ ]:
```
#Check null values of dataset
df.isnull().sum()
```

Out[ ]:
```
duration_sec                   0
start_time                     0
end_time                       0
start_station_id           80734
start_station_name         80133
start_station_latitude         0
start_station_longitude        0
end_station_id             82902
end_station_name           82350
end_station_latitude           0
end_station_longitude          0
bike_id                        0
user_type                      0
bike_share_for_all_trip   243259
rental_access_method     2386145
dtype: int64
```

Cleaning Data

There are roughly 3% of record of the dataset are nulls value for following columns `start_station_id`, `start_station_name`, `end_station_id`, `end_station_name` which I found that they are unfillable by any imputation. Therefore, will drop out these records from the dataset.

In [ ]:
```
# Drop null values for following columns
df.dropna(subset=['start_station_id', 'end_station_id', 'start_station_name',
```

In [ ]:
```
#Check null values dropped out of dataset for 4 columns: `start_station_id`, `s
df.isnull().sum()
```

Out[ ]:
```
duration_sec                   0
start_time                     0
end_time                       0
start_station_id               0
start_station_name             0
start_station_latitude         0
start_station_longitude        0
end_station_id                 0
end_station_name               0
end_station_latitude           0
end_station_longitude          0
bike_id                        0
user_type                      0
bike_share_for_all_trip   144280
rental_access_method     2385400
dtype: int64
```

Due to significant amount of null values for columns: `rental_access_method` & the dominant amount number of bike sharing is NO (more than 92%), it would be not valuable featuring analysis on this kind of project. Therefore, they will be dropped out of the dataset.

In [ ]:
```
#Amoun of sharing in the dataset
df.bike_share_for_all_trip.value_counts(normalize=True)*100
```

Out[ ]:
```
No     92.154722
Yes     7.845278
Name: bike_share_for_all_trip, dtype: float64
```

In [ ]:
```
#Drop 2 columns due to big amount of null values & being unusable
df.drop(['bike_share_for_all_trip', 'rental_access_method'], axis=1, inplace=T
```

In [ ]:
```
#Check 2 columns are removed from the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 2407259 entries, 0 to 2506660
Data columns (total 13 columns):
 #   Column                   Dtype
---  ------                   -----
 0   duration_sec             int64
 1   start_time               object
 2   end_time                 object
 3   start_station_id         float64
 4   start_station_name       object
 5   start_station_latitude   float64
 6   start_station_longitude  float64
 7   end_station_id           float64
 8   end_station_name         object
 9   end_station_latitude     float64
 10  end_station_longitude    float64
 11  bike_id                  int64
 12  user_type                object
dtypes: float64(6), int64(2), object(5)
memory usage: 257.1+ MB
```

There are following columns needed to be change to the right datatype:

- `start_time` & `end_time`: convert to datetime
- `start_station_id` & `end_station_id`: convert to object (string)
- `bike_id` & `user_type`: convert to category

In [ ]:
```
#Convert to datetime datatype
df['start_time'] = pd.to_datetime(df.start_time)
df['end_time'] = pd.to_datetime(df.end_time)
```

In [ ]:
```
#Checking the result
df[['start_time', 'end_time']].dtypes
```

Out[ ]:
```
start_time    datetime64[ns]
end_time      datetime64[ns]
dtype: object
```

In [ ]:
```
#Convert to string datatype
df['start_station_id'] = df['start_station_id'].astype('object')
df['end_station_id'] = df['start_station_id'].astype('object')
```

In [ ]:
```
#Checking the result
df[['start_station_id', 'start_station_id']].dtypes
```

Out[ ]:
```
start_station_id    object
start_station_id    object
dtype: object
```

In [ ]:
```
#Convert to category datatype
df['bike_id'] = df['bike_id'].astype('category')
df['user_type'] = df['user_type'].astype('category')
```

In [ ]:
```
#Checking the result

df[['bike_id', 'user_type']].dtypes
```

Out[ ]:
```
bike_id      category
user_type    category
dtype: object
```

In [ ]:
```
#Final after cleaning without null data, unusable columns & no duplicated reco
df.info(null_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 2407259 entries, 0 to 2506660
Data columns (total 13 columns):
 #   Column                   Non-Null Count    Dtype
---  ------                   --------------    -----
 0   duration_sec             2407259 non-null  int64
 1   start_time               2407259 non-null  datetime64[ns]
 2   end_time                 2407259 non-null  datetime64[ns]
 3   start_station_id         2407259 non-null  object
 4   start_station_name       2407259 non-null  object
 5   start_station_latitude   2407259 non-null  float64
 6   start_station_longitude  2407259 non-null  float64
 7   end_station_id           2407259 non-null  object
 8   end_station_name         2407259 non-null  object
 9   end_station_latitude     2407259 non-null  float64
 10  end_station_longitude    2407259 non-null  float64
 11  bike_id                  2407259 non-null  category
 12  user_type                2407259 non-null  category
dtypes: category(2), datetime64[ns](2), float64(4), int64(1), object(4)
memory usage: 227.6+ MB
```

Enriching features

- This dataset has timing feature in full format (day, month, year & time). Therefore, we can enrich more features to this dataset by adding columns:
  - time of day
  - day of the week
  - day of month
  - month of the year
  - quarter_of_year
  - is_weekend
- We convert the duration trip from second to minutes for better formatting of the data.

In [ ]:
```
# Enrich features from extracting time column
df['time_of_day'] = df['start_time'].dt.hour
```

```
df['day_of_week'] = df['start_time'].dt.day_name()
df['day_of_month'] = df['start_time'].dt.day
df['month_of_year'] = df['start_time'].dt.month
df['quarter_of_year'] = df['month_of_year'].apply(lambda x: ((x-1)//3 + 1))
df["is_weekend"] = df['start_time'].dt.day_name().isin(['Saturday', 'Sunday'])
```

In [ ]:
```
#Convert duration trip from second to minute
df['duration_min'] = round(df['duration_sec']/60, 1)
```

In [ ]:
```
df.sample(5)
```

Out[ ]:

	duration_sec	start_time	end_time	start_station_id	start_station_name	start_
108089	395	2019-01-17 15:10:31.163	2019-01-17 15:17:06.265	50.0	2nd St at Townsend St	
499084	353	2019-03-17 19:59:48.751	2019-03-17 20:05:42.615	97.0	14th St at Mission St	
546224	2195	2019-03-12 18:06:39.959	2019-03-12 18:43:15.339	18.0	Telegraph Ave at Alcatraz Ave	
512469	2201	2019-03-15 22:42:18.665	2019-03-15 23:19:00.597	157.0	65th St at Hollis St	
1896373	720	2019-09-06 09:32:24.435	2019-09-06 09:44:24.874	30.0	San Francisco Caltrain (Townsend St at 4th St)	

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 2407259 entries, 0 to 2506660
Data columns (total 20 columns):
 #   Column                   Dtype
---  ------                   -----
 0   duration_sec             int64
 1   start_time               datetime64[ns]
 2   end_time                 datetime64[ns]
 3   start_station_id         object
 4   start_station_name       object
 5   start_station_latitude   float64
 6   start_station_longitude  float64
 7   end_station_id           object
 8   end_station_name         object
 9   end_station_latitude     float64
 10  end_station_longitude    float64
 11  bike_id                  category
 12  user_type                category
 13  time_of_day              int64
 14  day_of_week              object
 15  day_of_month             int64
 16  month_of_year            int64
 17  quarter_of_year          int64
 18  is_weekend               bool
 19  duration_min             float64
dtypes: bool(1), category(2), datetime64[ns](2), float64(5), int64(5), object
(5)
memory usage: 340.1+ MB
```

In [ ]:
```
# Change datatype of enriching columns to correct one
df['time_of_day'] = df['time_of_day'].astype('object')
df['day_of_month'] = df['day_of_month'].astype('object')
df['month_of_year'] = df['month_of_year'].astype('object')
df['quarter_of_year'] = df['quarter_of_year'].astype('object')
```

In [ ]:
```
# Check the result
df[['time_of_day', 'day_of_month', 'month_of_year', 'quarter_of_year']].dtypes
```

Out[ ]:
```
time_of_day        object
day_of_month       object
month_of_year      object
quarter_of_year    object
dtype: object
```

What is the structure of your dataset?

> The dataset is included 2,407,259 records of bike trips of BayWheel Sanfrancisco in 2019. After enriching features, the dataset has total 20 columns. There are one main category feature, that is `user_type` (Subscriber or Customer). We need to find the the pattern usage of different type of user for better understanding their behavior on different time dimensions.

What is/are the main feature(s) of interest in your dataset?

> My analysis will focus on the usage behavior on time frame of different user types. From there, we can focus on pattern for better upgrading the service quality or promotion improvement to get them sastify and stay with the service as long as possible.

What features in the dataset do you think will help support your investigation into your feature(s) of interest?

> Many time dimension, trip duration & trip count would be the main features to support my investigation the the dataset.

Univariate Exploration

In [ ]:
```
# First copy the final cleaned & enriched dataframe
df_master = df.copy()
df_master.sample(5)
```

Out[ ]:

	duration_sec	start_time	end_time	start_station_id	start_station_name	start_
2468845	527	2019-12-03 17:42:08.031	2019-12-03 17:50:55.507	453.0	Brannan St at 4th St	
271293	210	2019-02-18 17:57:55.965	2019-02-18 18:01:26.246	76.0	McCoppin St at Valencia St	
2189009	677	2019-11-25 08:03:50.033	2019-11-25 08:15:07.594	468.0	Grove St at Gough St	
24854	557	2019-01-29 10:45:12.155	2019-01-29 10:54:29.680	171.0	Rockridge BART Station	
2306961	858	2019-11-07 17:05:16.432	2019-11-07 17:19:35.421	321.0	Folsom St at 5th St	

In [ ]:
```
#Save the df_master to the csv file called "2019_Baywheels-tripdata_master.csv"
df_master.to_csv('D:/_Data Analyst Stuff/DA_Udacity/Project 5/Final Project/201
```

Before to explore this dataset. There are couple points to get the overall information of the dataset:

- Total number of bikes
- Total number of start/end location

In [ ]:
```
#Total number of bikes
print(f"Total Number of bikes: {df_master.bike_id.nunique()}")
print(f"Total Number of start stations: {df_master.start_station_id.nunique()}"
print(f"Total Number of end stations: {df_master.start_station_id.nunique()}")
```

```
Total Number of bikes: 10766
Total Number of start stations: 434
Total Number of end stations: 434
```

In [ ]:
```
df_master.duration_min.describe().apply(lambda x: format(x, 'f'))
```

Out[ ]:
```
count    2407259.000000
mean          13.413596
std           31.845324
min            1.000000
25%            6.000000
50%            9.500000
75%           14.700000
max         1437.200000
Name: duration_min, dtype: object
```

The average trip duration is 13.4 minutes. The median trip duration is 9.5 minutes. The duration distribution is right-skewed where mean is larger than median. There are longest duration trips where last 1437.2 minutes. In contrast, the shortest one is just only 1 minute.

In [ ]:
```
# There is one trip that last 1437.2 min
df_master.query('duration_min == 1437.2')
```

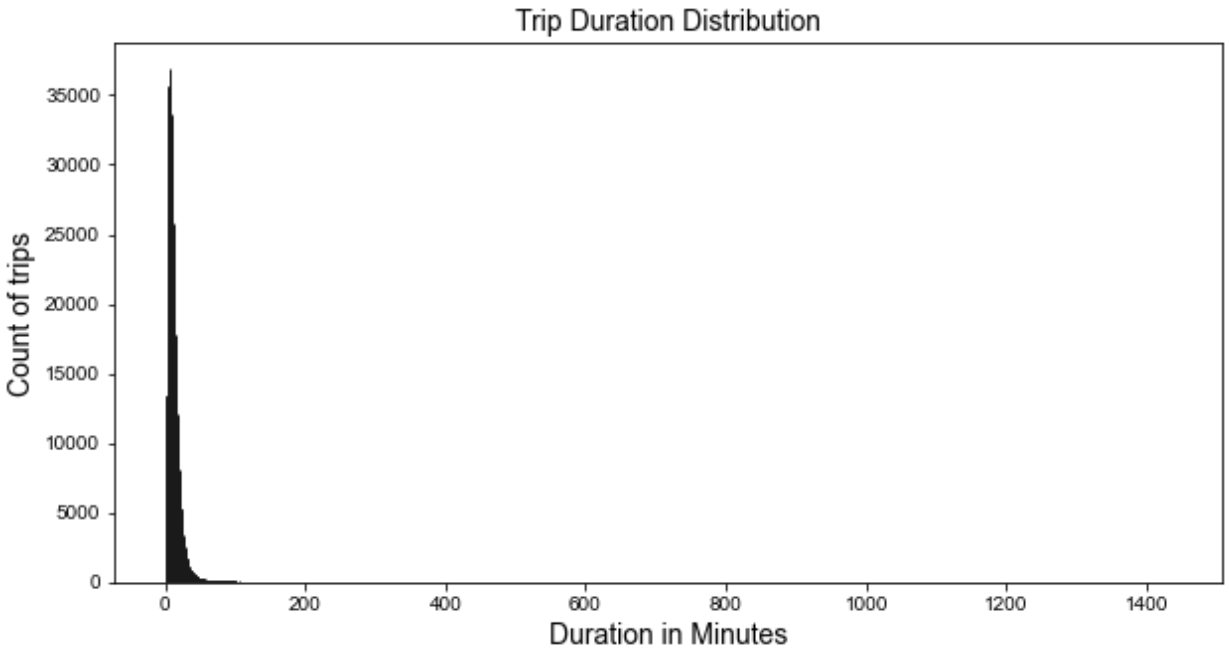
Out[ ]:

	duration_sec	start_time	end_time	start_station_id	start_station_name	start_s
2385537	86230	2019-12-18 08:35:17.307	2019-12-19 08:32:27.444	176.0	MacArthur BART Station	

In [ ]:
```
#Number of trips last 1 min
print(f"Number of trip last 1 minute: {df_master.query('duration_min == 1').sha
```

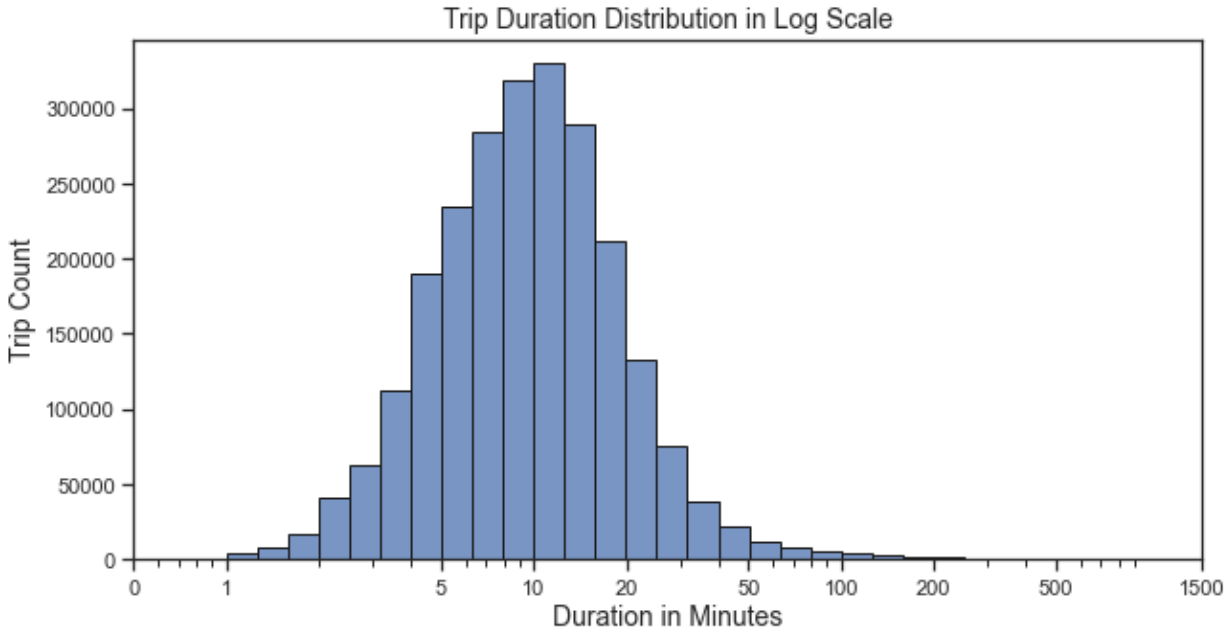
```
Number of trip last 1 minute: 802
```

In [ ]:
```
#Histogram of duration in minutes with linear scale
f1, ax1 = plt.subplots(figsize=(10, 5))
sns.set_style("whitegrid")
sns.set_theme(style="ticks")
sns.histplot(
    data=df_master,
    x="duration_min",
    edgecolor=".1",
    linewidth=1,
    bins=np.arange(0, df_master['duration_min'].max(), 0.1))
ax1.set_title('Trip Duration Distribution', size=14)
ax1.set_xlabel('Duration in Minutes', size=14)
ax1.set_ylabel('Count of trips', size=14);
```



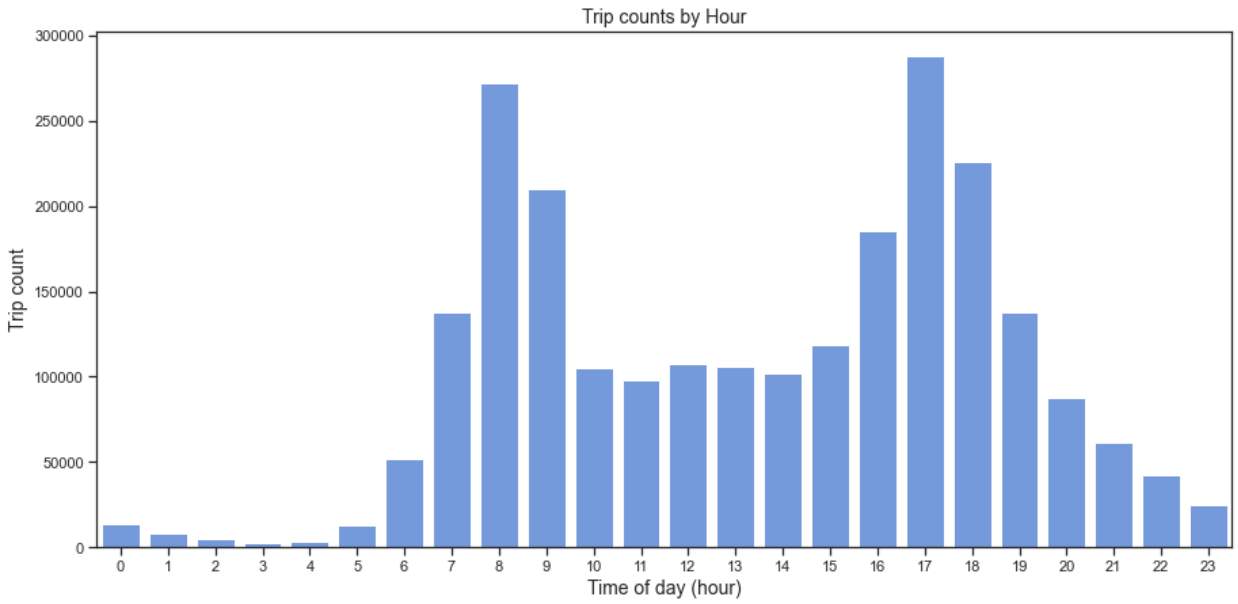
In [ ]:
```
#Histogram of duration in minutes in log scale
f1, ax1 = plt.subplots(figsize=(10, 5))
sns.set_style("whitegrid")
sns.set_theme(style="ticks")
sns.histplot(
    data=df_master,
    x="duration_min",
    edgecolor=".1",
    linewidth=1,
    log_scale=True,
    bins=np.arange(0, df_master['duration_min'].max(), 0.1)
)
ax1.xaxis.set_major_formatter(mpl.ticker.ScalarFormatter())
```

```
ax1.set_xticks([0.5, 1, 5, 10, 20, 50, 100, 200, 500, 1500])
ax1.set_title('Trip Duration Distribution in Log Scale', size=14)
ax1.set_xlabel('Duration in Minutes', size=14)
ax1.set_ylabel('Trip Count', size=14);
```



Observation: From the 2 histograms of trip duration in minutes in normal scale and log scale, we can see that most of the trip. Both of them is heavily skewed to the right on normal scale, but looks roughly a bell-shape distribution on log-scale. Most of the trips from 8 to 13 minutes. There are some trips with significant longer than 40min & less than 3 min.

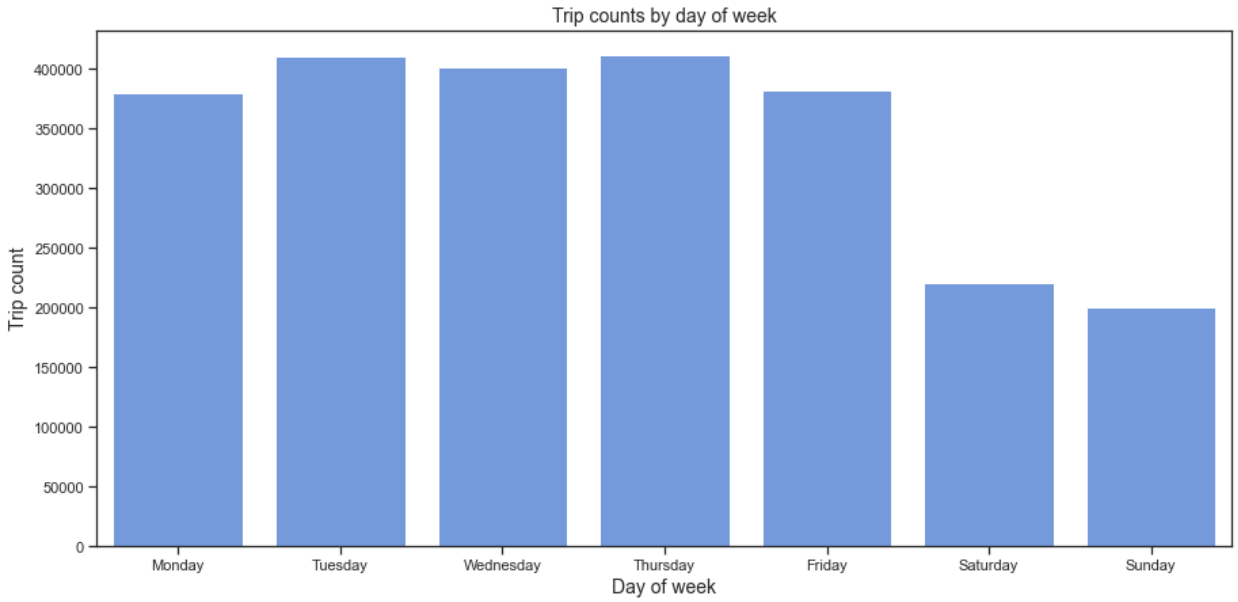
In [ ]:
```
sns.set_style("whitegrid")
sns.set_theme(style="ticks")
plt.figure(figsize=(15,7))
sns.countplot(x='time_of_day', data=df_master, color='cornflowerblue')
plt.title('Trip counts by Hour', size=14)
plt.xlabel('Time of day (hour)', size=14)
plt.ylabel('Trip count', size=14);
```



Observation: There are 2 peaks in the chart. One is on 8am the other is 5pm. This is the time of commuting to work in SF. Therefore, most of the trips are conducted to this time frame. Absolutely, the customer is from office workers. There is a pretty equally number of trips from 10am-3pm. They are gradually decreasing since 7pm till mid-night.

In [ ]:
```
day_of_week_ranked = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
day_of_week_ranked_class = pd.api.types.CategoricalDtype(ordered=True, categor
df_master['day_of_week'] = df_master['day_of_week'].astype(day_of_week_ranked_

sns.set_style("whitegrid")
sns.set_theme(style="ticks")
plt.figure(figsize=(15,7))
sns.countplot(x='day_of_week', data=df_master, color='cornflowerblue')
plt.title('Trip counts by day of week',size=14)
plt.xlabel('Day of week',size=14)
plt.ylabel('Trip count',size=14);
```



Observation: The big number of trips is on weekday (working day). It significant reduces on weekend when office workers absolutely off from work. As a result, they are not using the bikes

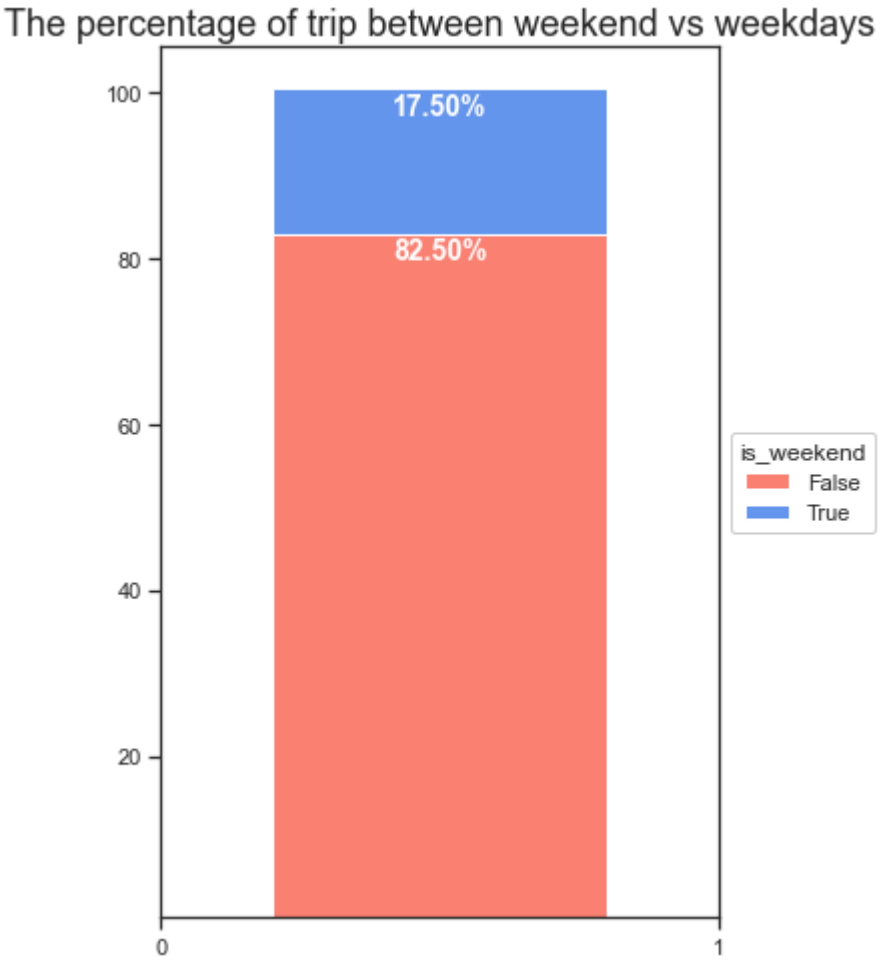
In [ ]:
```
# Percentage of weekend vs weekday trip
sorted_counts = df_master['is_weekend'].value_counts(normalize=True)*100
plt.figure(figsize = [5, 8])
base_color = sns.color_palette()[:3]
colors = ['salmon', 'cornflowerblue']

baseline = 0.5
for i in range(sorted_counts.shape[0]):
    ax = plt.bar(x = 0.5, height = sorted_counts[i], bottom = baseline, color=
    baseline += sorted_counts[i]
    plt.annotate(f'{sorted_counts[i]:.2f}%',
                 (0.5, sorted_counts[:i].sum()+sorted_counts[i] ),
                 color='white', fontsize=14, weight='bold', ha='center', va=

plt.xticks(np.arange(len(sorted_counts)))
plt.legend(sorted_counts.index, title='is_weekend', framealpha = 1,
```

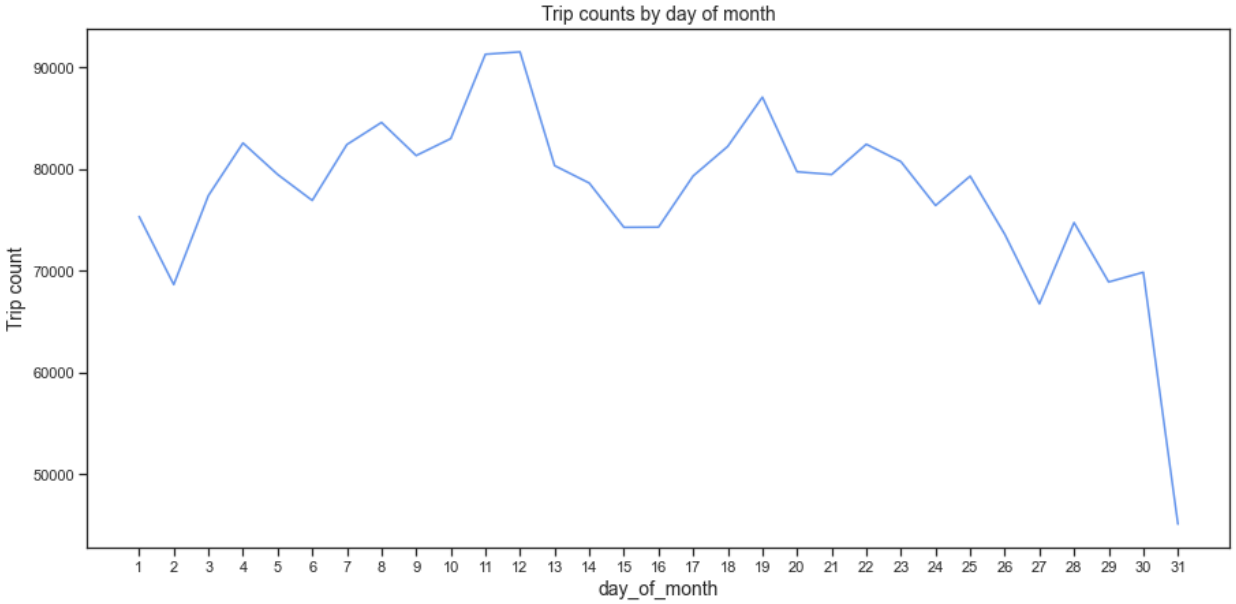
```
            bbox_to_anchor = (1, 0.5), loc = 6);  # add a legend for labeling

plt.title('The percentage of trip between weekend vs weekdays', fontsize=18);
```



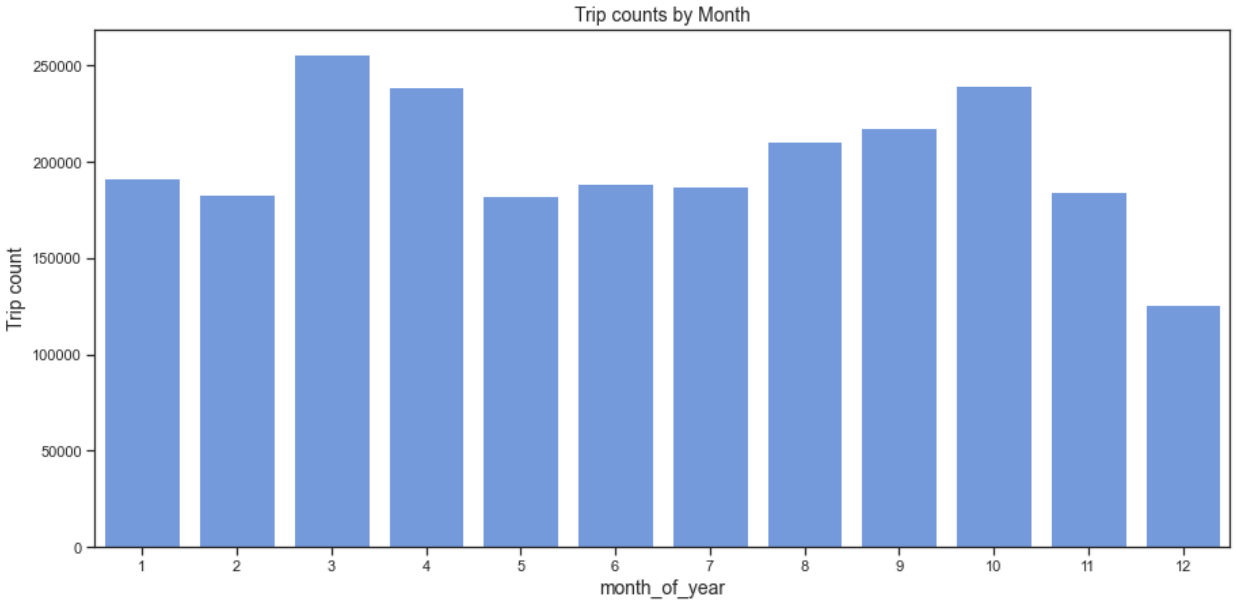
Observation: More than 80 percent of trip is conducted on weekday rather than weekend as a confirmation again the above statement from previous distribution trip is on weekdays.

In [ ]:
```
sns.set_style("darkgrid")
sns.set(style="ticks")
plt.figure(figsize=(15,7))
sns.lineplot(data=df_master['day_of_month'].value_counts(), color='cornflowerb
plt.xticks(np.arange(1,32,1))
plt.title('Trip counts by day of month',size=14)
plt.xlabel('day_of_month',size=14)
plt.ylabel('Trip count',size=14);
```



Observation: The peak number of trip is on every 11st, 12nd of the month. Surprisingly, this number significant drops on middile of the month and come back increases on the end of the month. However, the lowest point is on 27th of every month. This could assume that people tend to not riding bikes on mid-month and some time end of the month. We can take action to boost their activity to re-gain the number back on track on these time-frame.

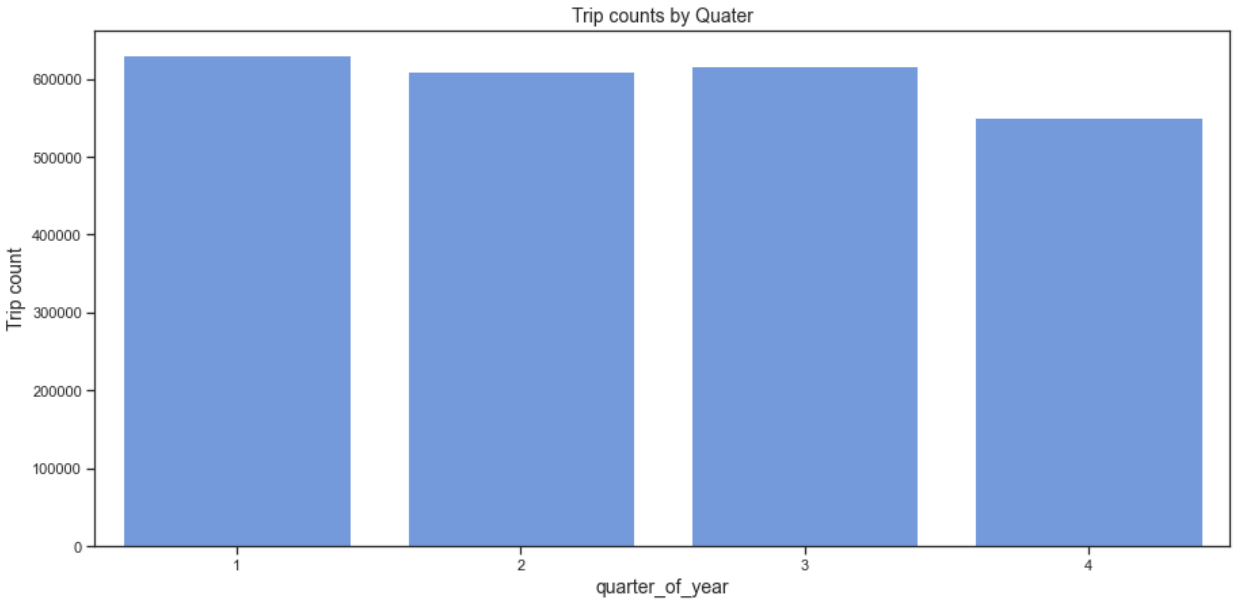
In [ ]:
```
sns.set_style("whitegrid")
sns.set_theme(style="ticks")
plt.figure(figsize=(15,7))
sns.countplot(x='month_of_year', data=df_master, color='cornflowerblue')
plt.title('Trip counts by Month',size=14)
plt.xlabel('month_of_year',size=14)
plt.ylabel('Trip count',size=14);
```



Observation: Spring time (March, April) & autumn time (September, October) are the peak months. This absolutely relevant to the seasonality. On winter time whose the temperature

is low and summer time whose temperature is high, they could affect to the riders behaviors. These reflect to the number of trips are lower than spring & autumn time.

In [ ]:
```
sns.set_style("whitegrid")
sns.set_theme(style="ticks")
plt.figure(figsize=(15,7))
sns.countplot(x='quarter_of_year', data=df_master, color='cornflowerblue')
plt.title('Trip counts by Quater',size=14)
plt.xlabel('quarter_of_year',size=14)
plt.ylabel('Trip count',size=14);
```



Observation: The number of trips is gradually reducing from quarter to quarter. However, on quarter 4, this has a drop that could be the holiday season when office workers are off from work.

In [ ]:
```
# Percentage of user type
user_type_relative = df_master['user_type'].value_counts(normalize=True)*100


# plt.figure(figsize = [5, 8])
# base_color = sns.color_palette()[:3]
colors = ['cornflowerblue' , 'salmon']

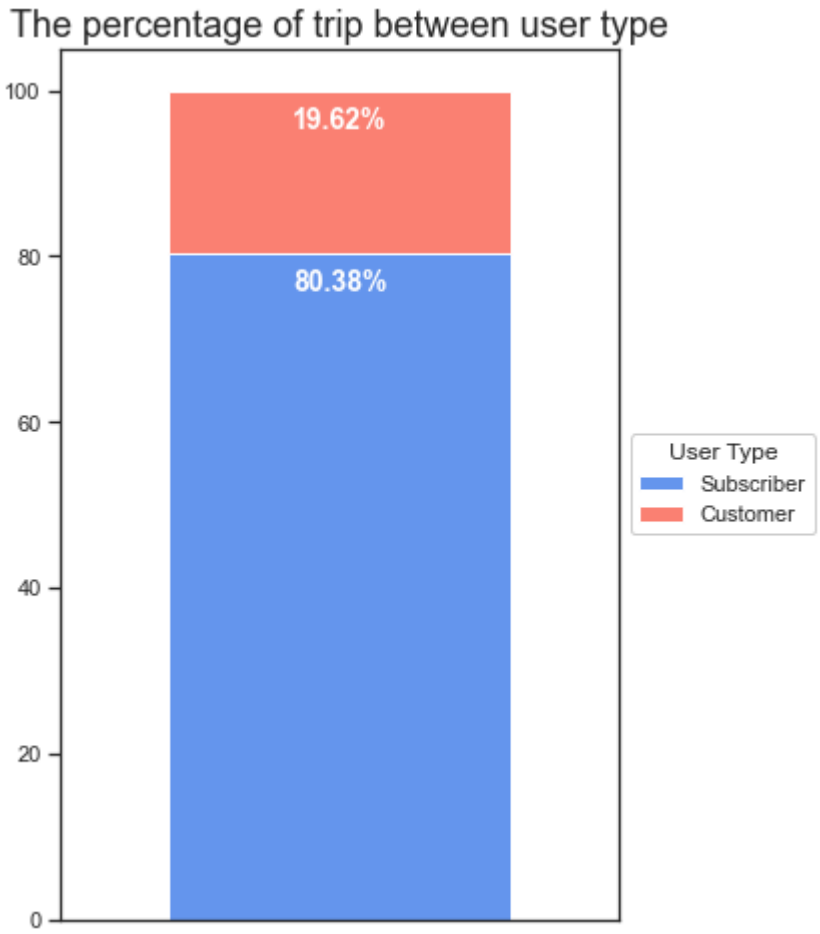
ax = user_type_relative.to_frame().T.plot.bar(stacked=True, color=colors, figs
adjust = -2

for bars in ax.patches:
    adjust += bars.get_height()
    plt.annotate(f'{bars.get_height():.2f}%',
                 (bars.get_x()+bars.get_width()/2, adjust),
                 color='white', fontsize=14, weight='bold', ha='center', va

# baseline = 0.5
# for i in range(user_type_relative.shape[0]):
#     ax = plt.bar(x = 0.5, height = user_type_relative[i], bottom = baseline,
#     baseline += user_type_relative[i]
#     plt.annotate(f'{user_type_relative[i]:.2f}%',
#                  (0.5, user_type_relative[:i].sum()+user_type_relative[i]
```

```
#                     color='white', fontsize=14, weight='bold', ha='center', v
plt.legend(user_type_relative.index, title='User Type', framealpha = 1,
           bbox_to_anchor = (1, 0.5), loc = 6);  # add a legend for labeling
plt.xticks([])

plt.title('The percentage of trip between user type', fontsize=18);
```



Observation: There is approximately more than 80% of trips is from subscriber rather than customer

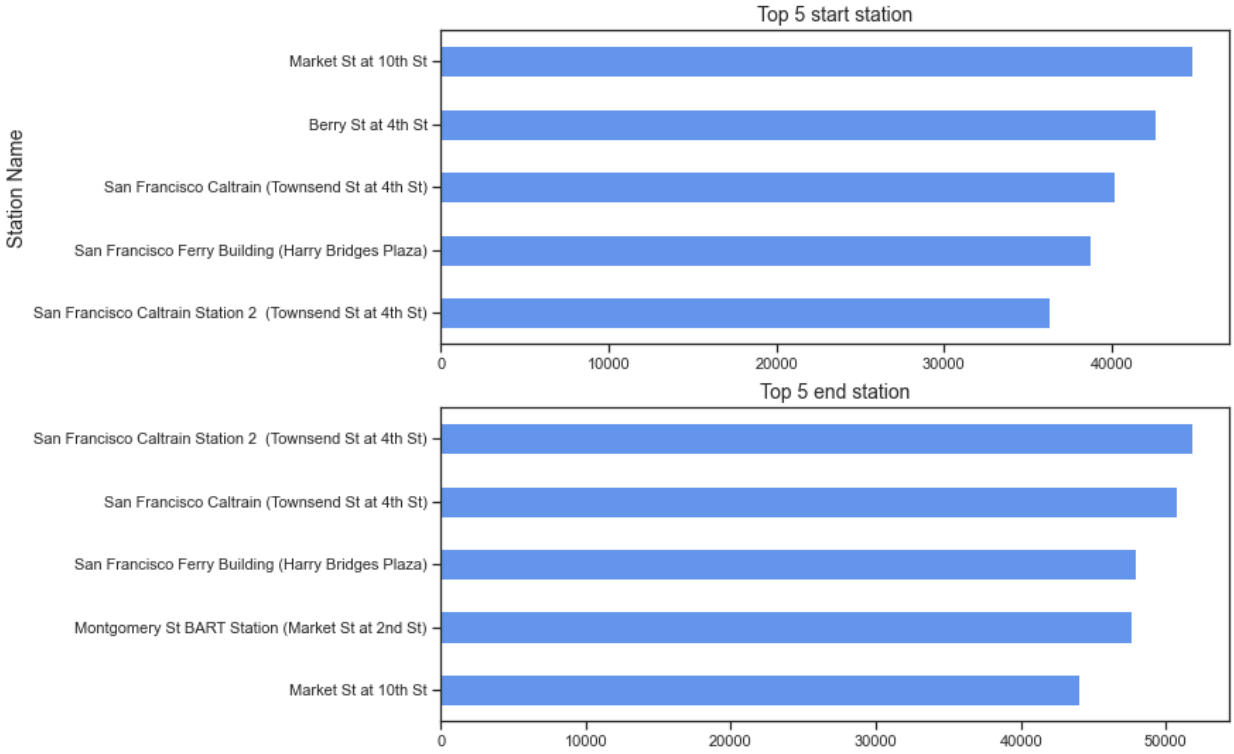
In [ ]:
```
user_type_relative
```

Out[ ]:
```
Subscriber    80.383623
Customer      19.616377
Name: user_type, dtype: float64
```

In [ ]:
```
plt.figure(figsize=(10,9))

plt.subplot(2, 1, 1)
top_5_start_station = df_master['start_station_name'].value_counts()
top_5_start_station.iloc[:5].sort_values(ascending=True).plot(kind='barh', colo
plt.title("Top 5 start station", size=14)
plt.ylabel("Station Name", size=14);

plt.subplot(2, 1, 2)
top_5_end_station = df_master['end_station_name'].value_counts()
top_5_end_station.iloc[:5].sort_values(ascending=True).plot(kind='barh', color=
plt.title("Top 5 end station", size=14);
```



Observation: 4 out of 5 stations are on the same top 5 start/end station: Market St at 10th St, SF Caltrain(Townsend St at 4th St), SF Ferry Building (Harry Bridges Plaza), SF Caltrain Station 2 (Townsend St at 4th St).

Discuss the distribution(s) of your variable(s) of interest. Were there any unusual points? Did you need to perform any transformations?

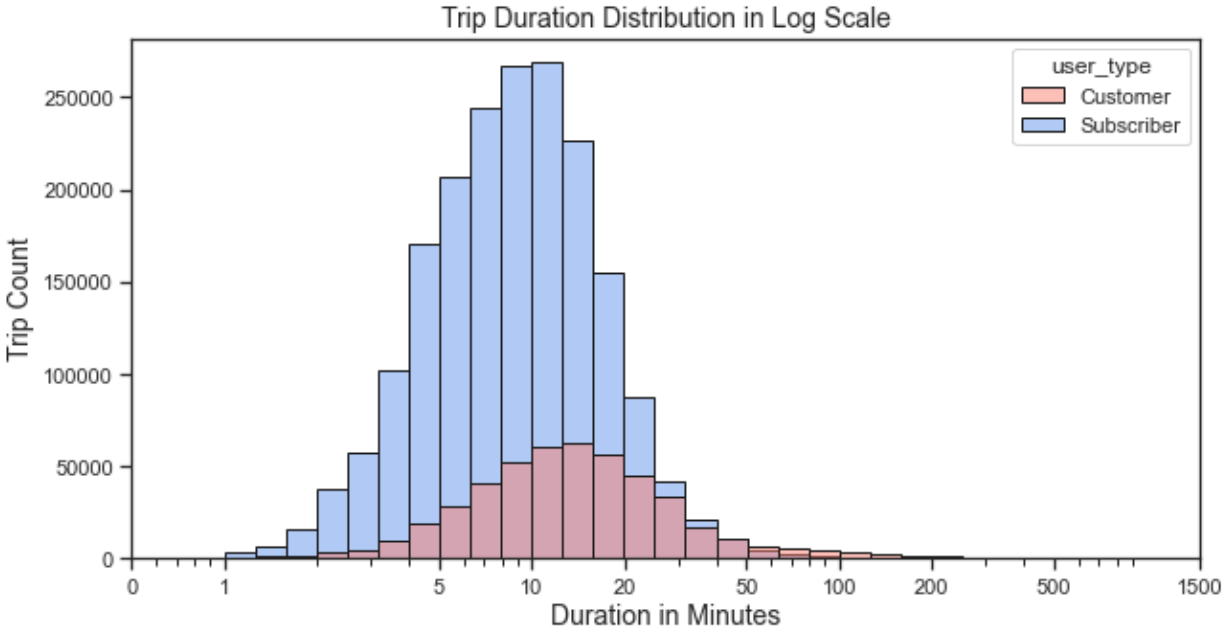
> - The trip duration in minutes is in such a wide range. Therefore, the I switch to use the log-scale for better visualization. It is skewed to the right. And it looks like unimodal distribution with the peak around 12 minutes.
> - Interestingly, there are 2 peaks in time of day 8am & 5pm. Most of the trip conducted.
> - There is a significant drop of trips on weekends compared to weekday.
> - 82.5% of the trip is in weekday.
> - 80.3% of the trip is from Subscribers.
> - There is a drop in trip in mid-month from every 13rd to 18th of the month
> - March & October are two months where the peaks are.

Of the features you investigated, were there any unusual distributions? Did you perform any operations on the data to tidy, adjust, or change the form of the data? If so, why did you do this?

> Seem the outliers of the dataset in duration is valid point. We consider to keep it for analysis.

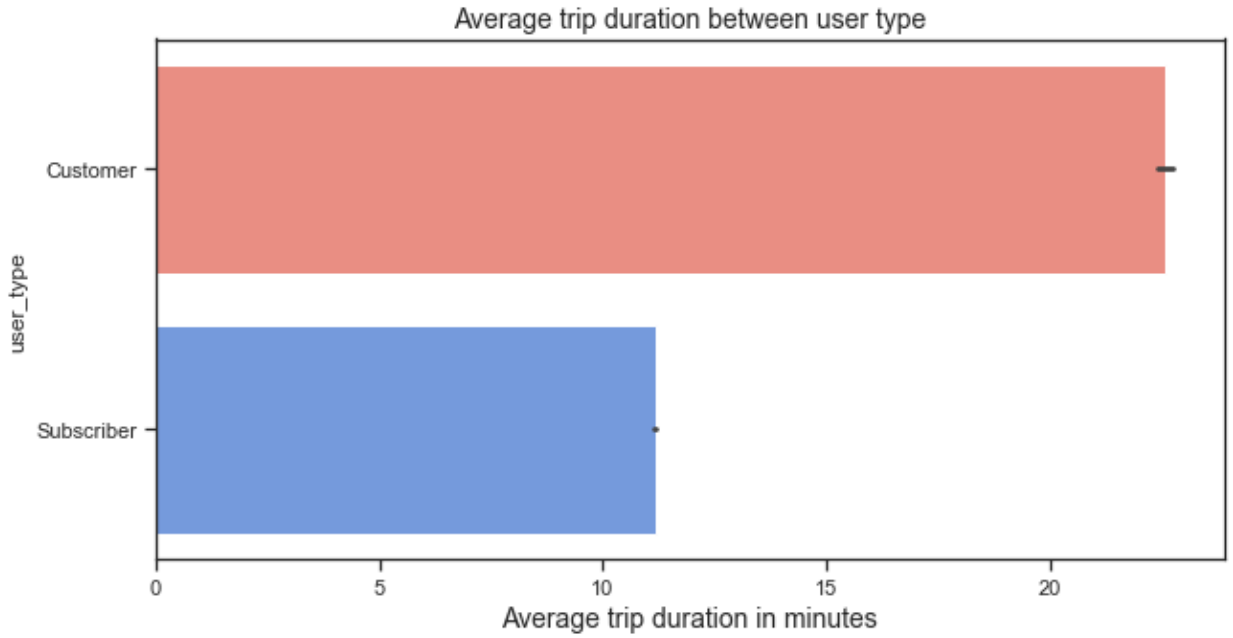
Bivariate Exploration

In [ ]:
```
#Histogram of duration in minutes in log scale
f1, ax1 = plt.subplots(figsize=(10, 5))
sns.set_style("whitegrid")
sns.set_theme(style="ticks")
sns.histplot(
    data=df_master,
    x="duration_min",
    edgecolor=".1",
    linewidth=1,
    hue='user_type',
    log_scale=True,
    bins=np.arange(0, df_master['duration_min'].max(), 0.1),
    palette= ['salmon', 'cornflowerblue'])
ax1.xaxis.set_major_formatter(mpl.ticker.ScalarFormatter())
ax1.set_xticks([0.5, 1, 5, 10, 20, 50, 100, 200, 500, 1500])
ax1.set_title('Trip Duration Distribution in Log Scale', size=14)
ax1.set_xlabel('Duration in Minutes', size=14)
ax1.set_ylabel('Trip Count', size=14);
```



Observation: The overlay distribution of 2 user types indicates that Subscriber contribute the most of the trips. However, customer trip duration distribution is typically on 10-20 minutes that is longer than subscribers who rides mostly from 8 minutes to 10 minutes.

In [ ]:
```
fig, axes = plt.subplots(figsize=(10, 5))
sns.barplot(y=df_master['user_type'], x=df_master['duration_min'], palette= ['
axes.set_title("Average trip duration between user type", size=14)
axes.set_xlabel("Average trip duration in minutes", size=14);
```



In [ ]:
```
df_master.groupby('user_type')['duration_min'].mean().reset_index()
```

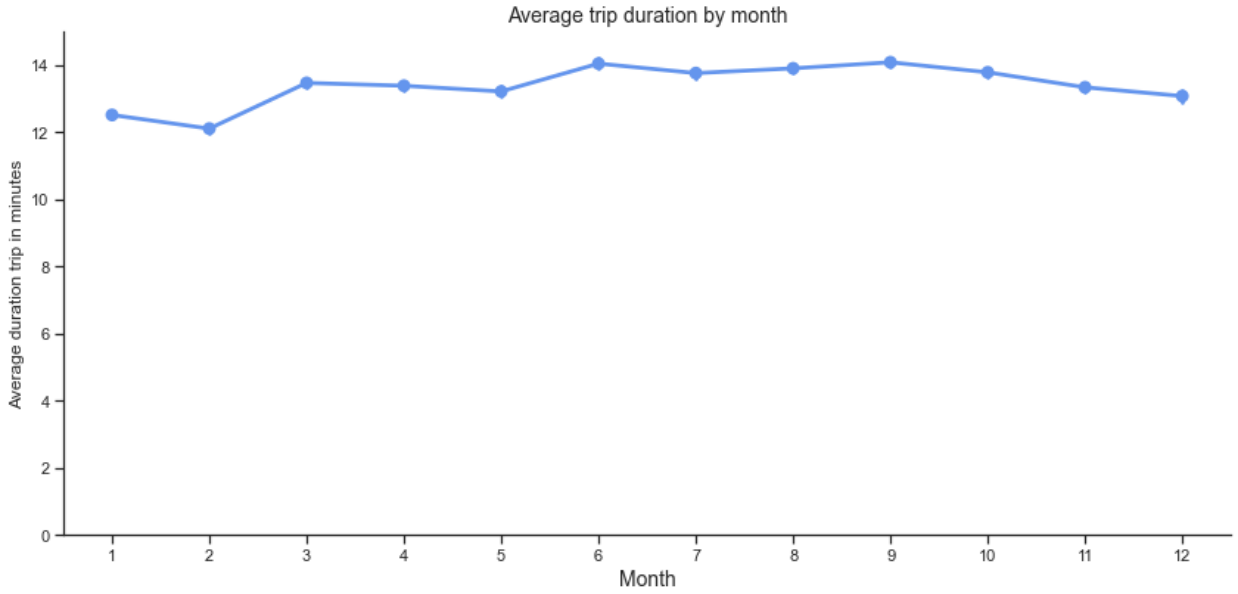
Out[ ]:

	user_type	duration_min
0	Customer	22.579533
1	Subscriber	11.176791

Observation: Although the number of trip conducted by customer is far smaller than subscriber, their average trip duration (22 minutes) is double from subscriber (11 minutes).

In [ ]:
```
sns.catplot(data=df_master,
            y="duration_min",
            x = "month_of_year",
            kind= "point",
            aspect=2,
            height=6,
            dodge = 0.3,
            color='cornflowerblue');

plt.xlabel("Month", size=14)
plt.ylabel("Average duration trip in minutes")
plt.title("Average trip duration by month", size=14)
plt.ylim([0, 15]);
plt.tight_layout(pad=1.5);
```



In [ ]:
```
average_duration_by_month = df_master.groupby('month_of_year')['duration_min']
average_duration_by_month.to_frame(name='average_trip_duration (min)')
```

Out[ ]:

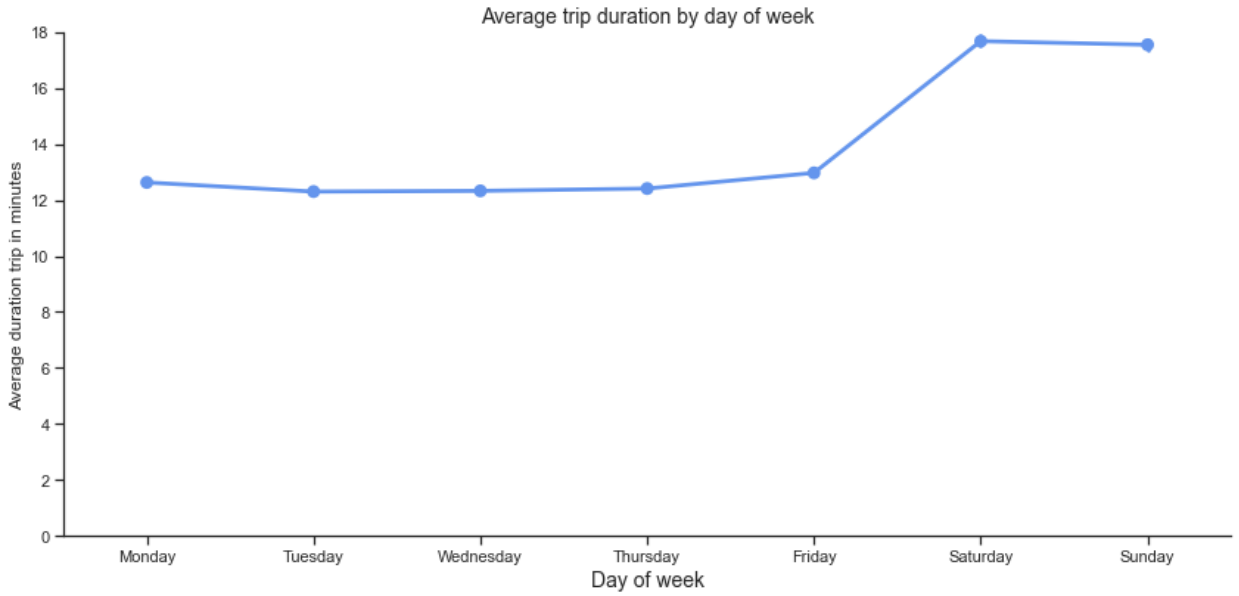
month_of_year	average_trip_duration (min)
1	12.504297
2	12.098280
3	13.458497
4	13.374096
5	13.204231
6	14.033813
7	13.749925
8	13.891898
9	14.072612
10	13.773611
11	13.327152
12	13.066769

Observation: Average trip duration in 2019 is 13.41 min. From this plot, we can see that the lowest trip duration is on Feb/2019. And the highest duration is on Sep/2019. The trend is a bit gradually dropping to the end of the month. It could be the holiday season, and the weather is getting cold that could affect on riding behavior.

In [ ]:
```
sns.catplot(data=df_master,
            y="duration_min",
            x = "day_of_week",
            kind= "point",
            aspect=2,
            height=6,
```

```
            dodge = 0.3,
            color='cornflowerblue');

plt.xlabel("Day of week", size=14)
plt.ylabel("Average duration trip in minutes")
plt.title("Average trip duration by day of week", size=14)
plt.ylim([0, 18]);
plt.tight_layout(pad=1.5);
```



In [ ]:
```
average_duration_by_day_of_week = df_master.groupby('day_of_week')['duration_m
average_duration_by_day_of_week.to_frame(name='average_trip_duration (min)')
```

Out[ ]:

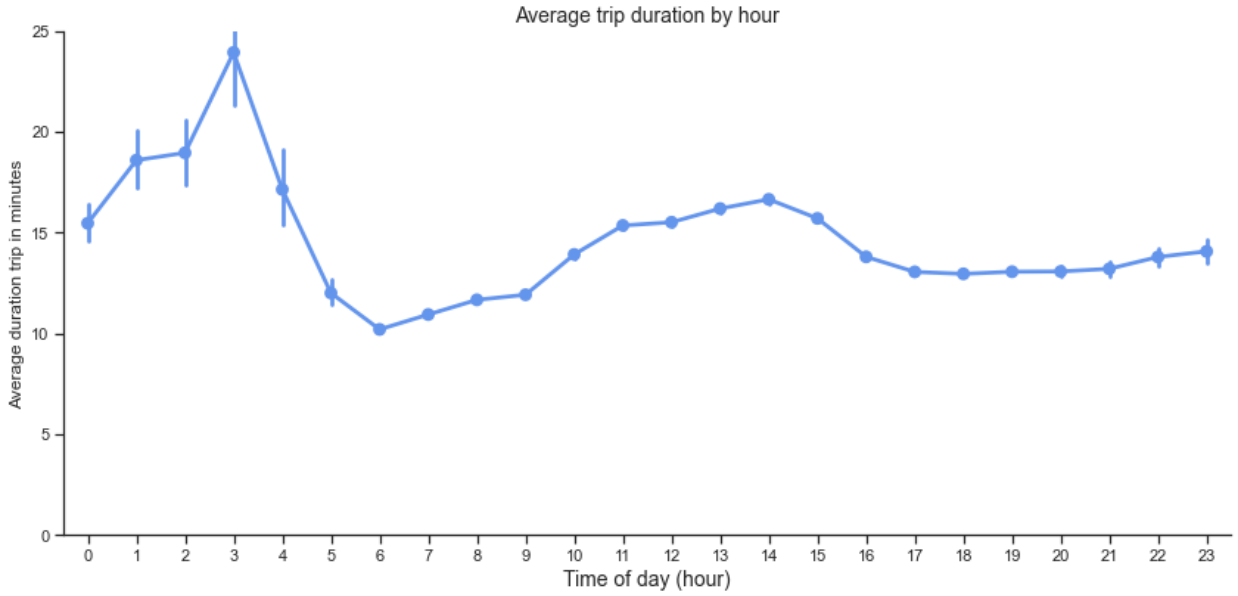
day_of_week	average_trip_duration (min)
Monday	12.629375
Tuesday	12.303137
Wednesday	12.328849
Thursday	12.412246
Friday	12.971128
Saturday	17.677261
Sunday	17.549196

Observation: Average trip duration in 2019 is 13.41 min. The number of trips on Saturday & Sunday is lower compared to other days in weekdays. However, there is a big difference in the trip duration on weekends as the above plot that show that people usually ride about 4-5 min longer than weekdays.

In [ ]:
```
sns.catplot(data=df_master,
            y="duration_min",
            x = "time_of_day",
            kind= "point",
            aspect=2,
```

```
            height=6,
            dodge = 0.3,
            color='cornflowerblue'
            );

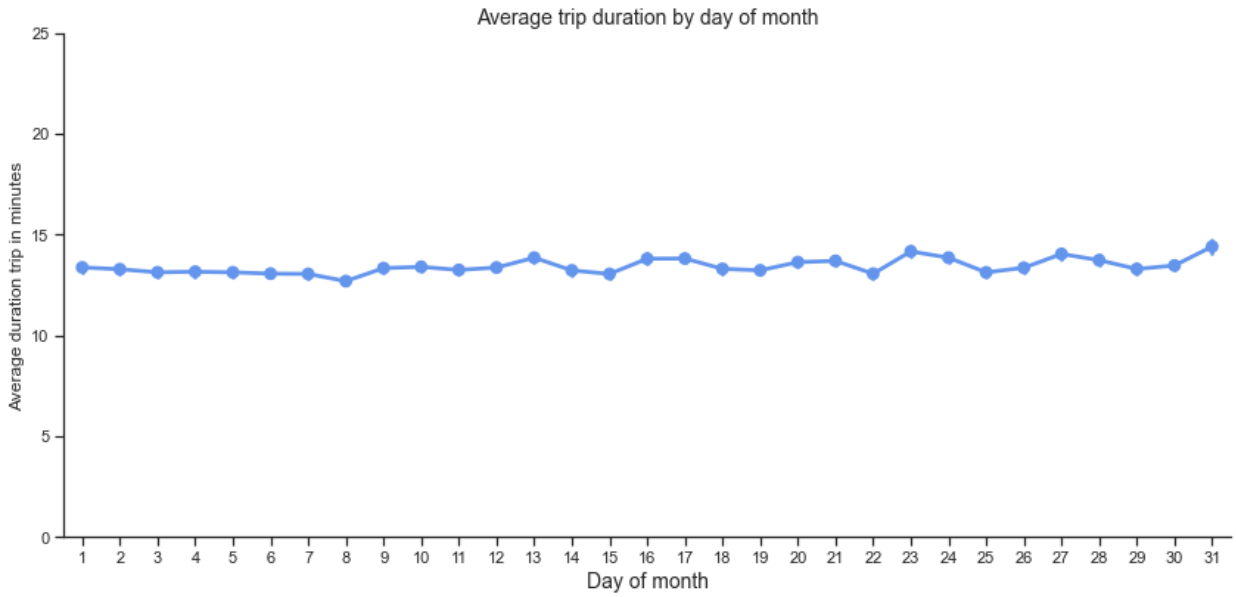
plt.xlabel("Time of day (hour)", size=14)
plt.ylabel("Average duration trip in minutes")
plt.title("Average trip duration by hour", size=14)
plt.ylim([0, 25]);
plt.tight_layout(pad=1.5);
```



Observation: The average trip duration since mid-night to 3am has been higher but the values vary a lot showed it by the longer errors bars due to fewer trips occurring at this timeframe. The trend then drops significantly till morning and get stable till mid-day with least variation compared to other time frames within the day. It absolutely people tend to do investigate the behavior of longer trip duration in late-night to early morning for better understanding the pattern here.

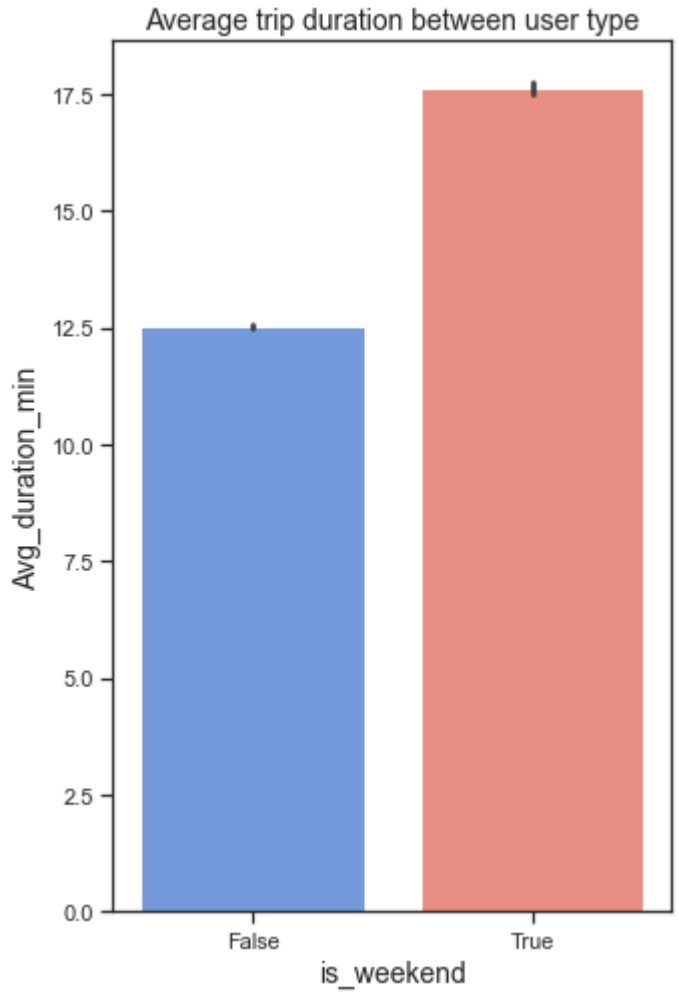
In [ ]:
```
sns.catplot(data=df_master,
            y="duration_min",
            x = "day_of_month",
            kind= "point",
            aspect=2,
            height=6,
            dodge = 0.3,
            color='cornflowerblue'
            );

plt.xlabel("Day of month", size=14)
plt.ylabel("Average duration trip in minutes")
plt.title("Average trip duration by day of month", size=14)
plt.ylim([0, 25]);
plt.tight_layout(pad=1.5);
```



Observation: There is not so much different in average trip duration among days of month.

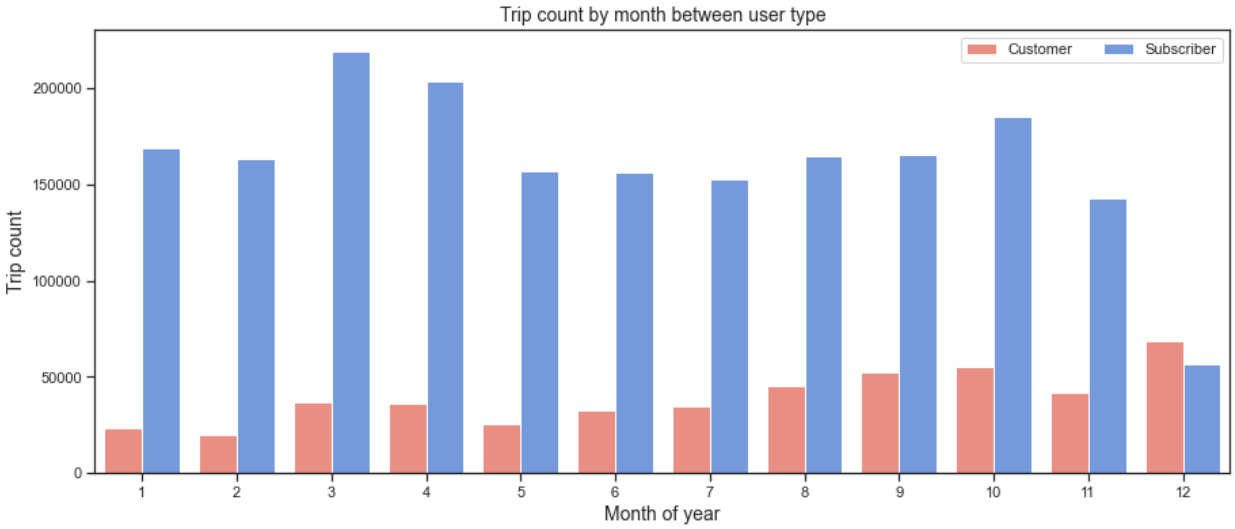
In [ ]:
```
fig, axes = plt.subplots(figsize=(5, 8))
sns.barplot(data=df_master, y='duration_min', x='is_weekend', palette=['cornfl
axes.set_title("Average trip duration between user type", size=14)
axes.set_ylabel("Avg_duration_min", size=14)
axes.set_xlabel("is_weekend", size=14);
```



Observation: The weekend riders spend more time to ride the bike rather than on weekdays rider, inspite of lower the number of weekend riders than weekday riders with 5 minutes difference.

In [ ]:
```
fig, axes = plt.subplots(figsize=(15, 6))

sns.countplot(data = df_master, x = 'month_of_year', hue = 'user_type', palett
axes.legend(loc = 1, ncol = 2)
axes.set_title("Trip count by month between user type", size=14)
axes.set_ylabel("Trip count", size=14)
axes.set_xlabel("Month of year", size=14);
```

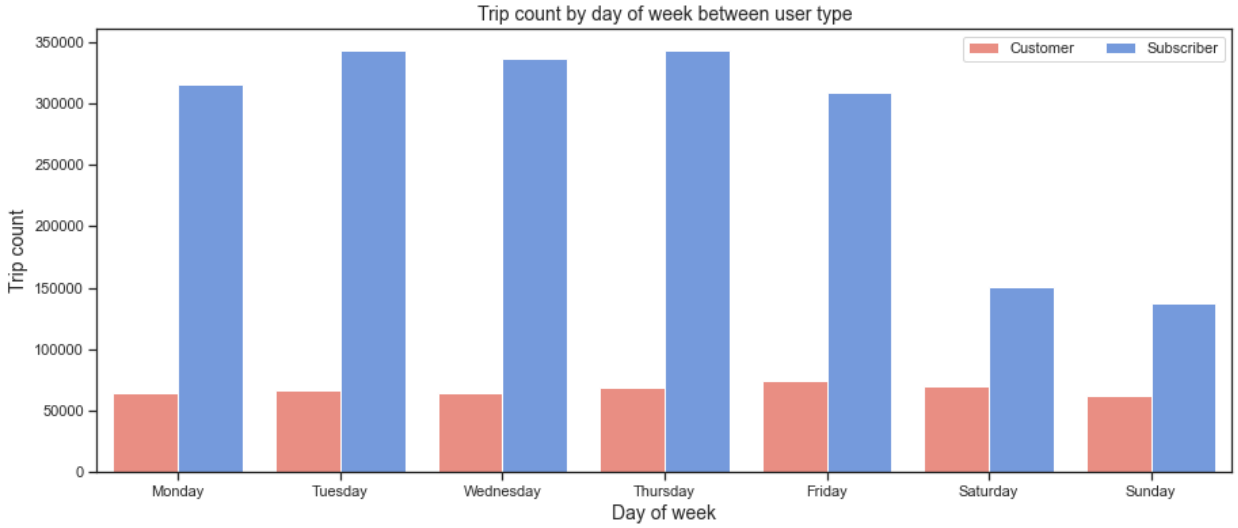


Observation:

- Overall, the subscriber contributes the greatest number of the trip over the months of 2019. However, in December, trip count's customer is higher than subscriber. This could be because of the holiday season so our regular bikers are not going to commute to work (perhaps they take annual leaves) rather than the customer could be tourists or travelers from outside of the city. -The month of using for subscriber is in March and April. After that the number of trips is dropping as coming to summertime and come back on springtime (September & October). And it significant dops on holiday season when most of office workers is taking time off from work and not commuting to work.
    - On the other hand, customer starts using more bikes from September till end of the year.

In [ ]:
```
fig, axes = plt.subplots(figsize=(15, 6))

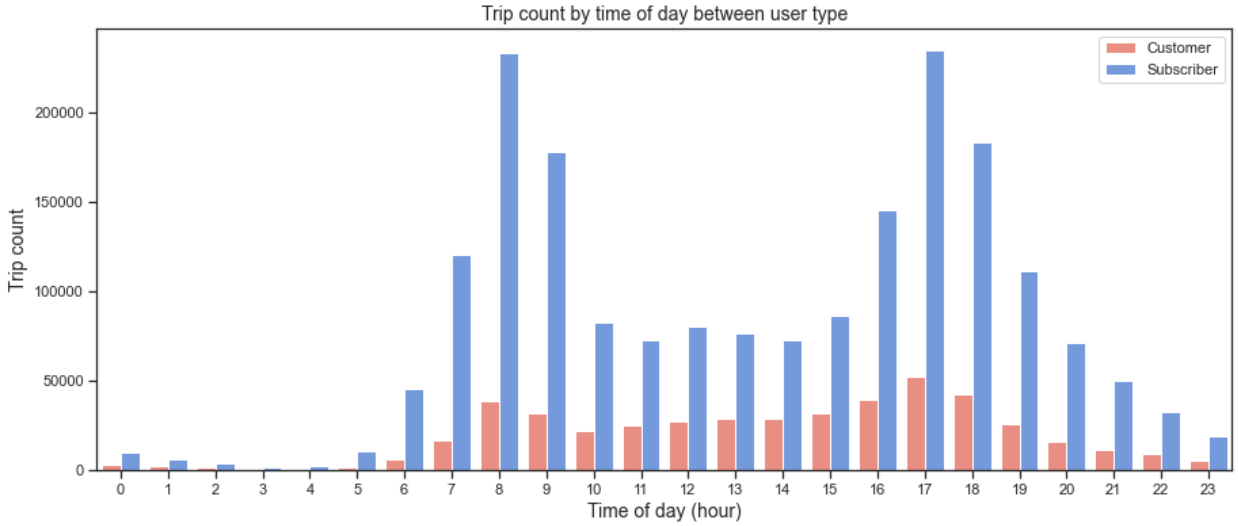
sns.countplot(data = df_master, x = 'day_of_week', hue = 'user_type', palette=
axes.legend(loc = 1, ncol = 2)
axes.set_title("Trip count by day of week between user type", size=14)
axes.set_ylabel("Trip count", size=14)
axes.set_xlabel("Day of week", size=14);
```



Observation: There is a sharp decrease in the number of trip on weekends from subscribers starting from Friday. For customer the trip number is more stable over the week.

In [ ]:
```
fig, axes = plt.subplots(figsize=(15, 6))

sns.countplot(data = df_master, x = 'time_of_day', hue = 'user_type', palette=
axes.legend(loc = 1, ncol = 1)
axes.set_title("Trip count by time of day between user type", size=14)
axes.set_ylabel("Trip count", size=14)
axes.set_xlabel("Time of day (hour)", size=14);
```

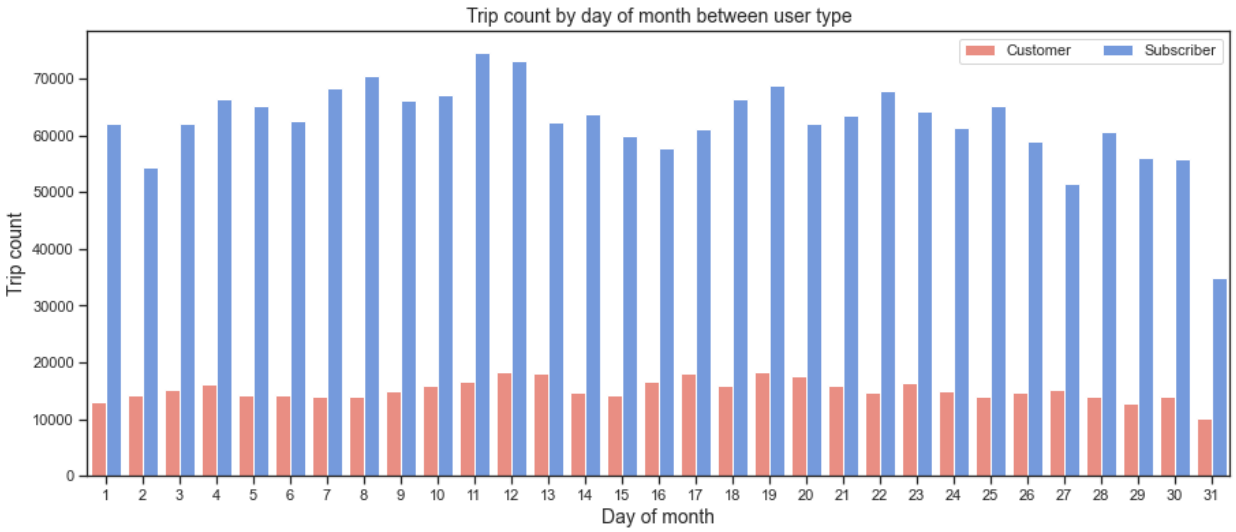


Observation: Subscriber riders use the bikes the the time frame of 7-9am & 4-6pm. Absolutely, the time of commuting to work. On the other hand, customer ride the bike starting morning till evening and the trip is stablizing over the time and the peak is on 5pm of the day then off bikes till night time. Interestingly, some subscribers use the bike on very early morning from mid-night till 2am.

In [ ]:
```
fig, axes = plt.subplots(figsize=(15, 6))

sns.countplot(data = df_master, x = 'day_of_month', hue = 'user_type', palette=
axes.legend(loc = 1, ncol = 2)
axes.set_title("Trip count by day of month between user type", size=14)
```

```
axes.set_ylabel("Trip count", size=14)
axes.set_xlabel("Day of month", size=14);
```

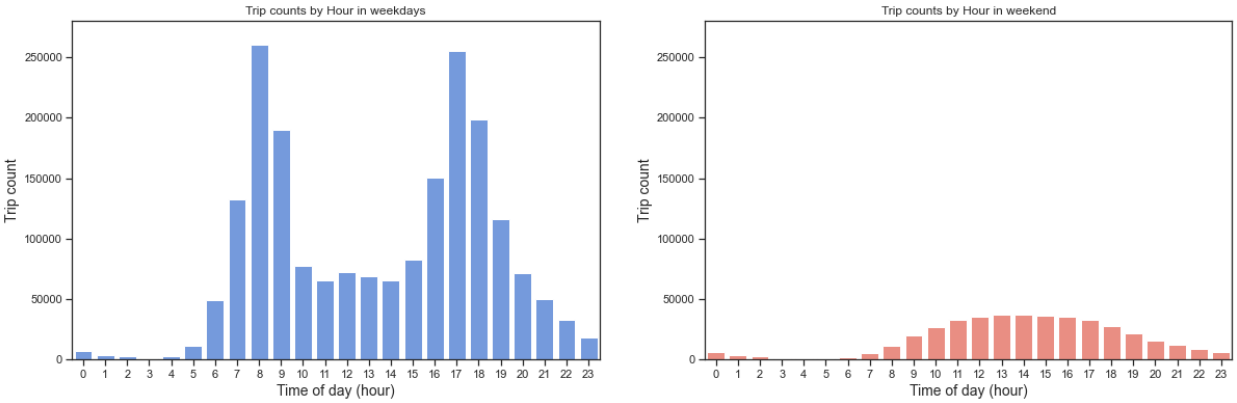


Observation: The customer use the bike more stably over the day in month. The subscribers use the bike is far higher than the customer however they are noting using very much on mind month and end of the month.

In [ ]:
```
plt.figure(figsize=(20, 6))

plt.subplot(1, 2, 1)
sns.countplot(x='time_of_day', data=df_master[df_master['is_weekend'] == False
plt.title('Trip counts by Hour in weekdays')
plt.xlabel('Time of day (hour)', size=14)
plt.ylabel('Trip count', size=14)
plt.ylim([0, 280000])


plt.subplot(1, 2, 2)
sns.countplot(x='time_of_day', data=df_master[df_master['is_weekend'] == True]
plt.title('Trip counts by Hour in weekend')
plt.xlabel('Time of day (hour)', size=14)
plt.ylabel('Trip count', size=14)
plt.ylim([0, 280000]);
```



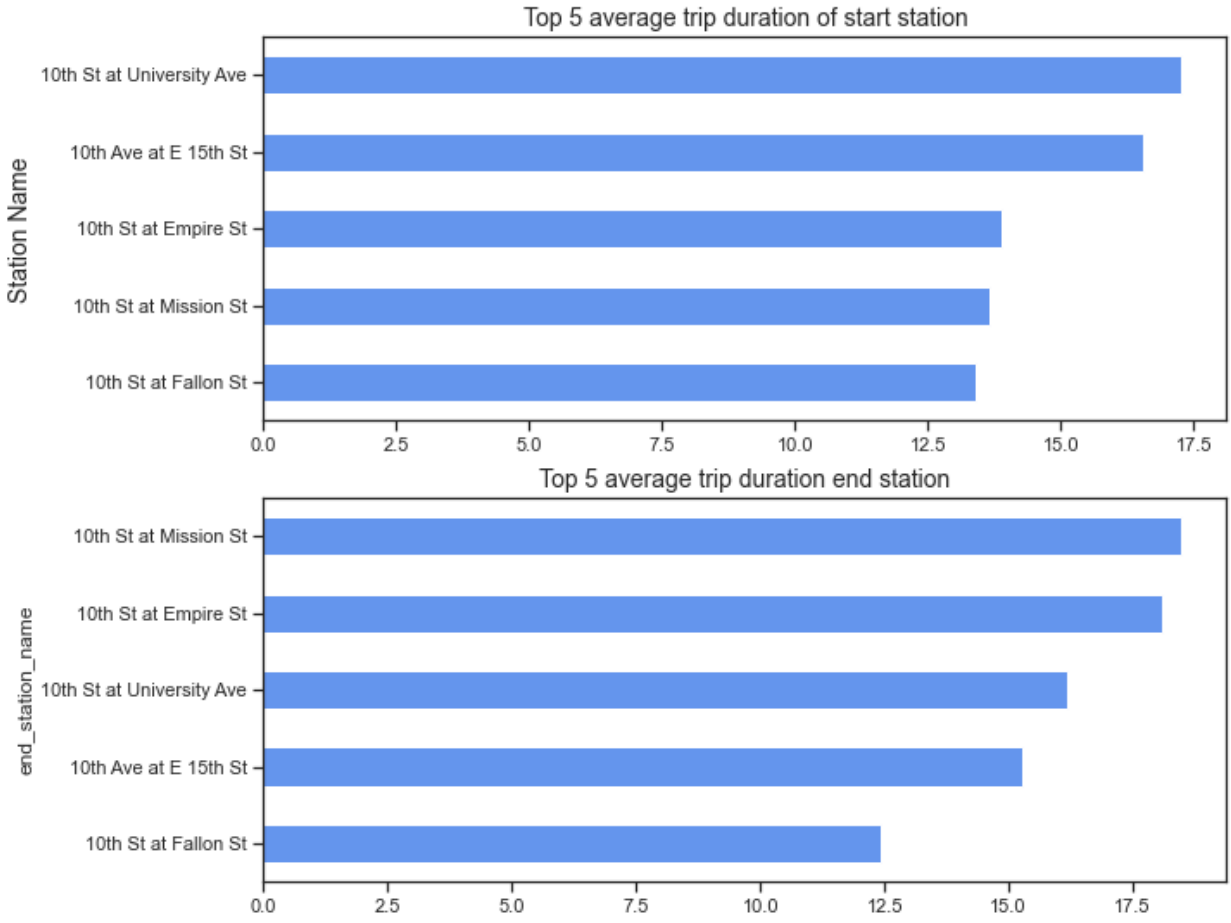
Observation: On weekdays trips are mostly occurs on the time frame of commuting to work of office workers. As a result, most of the trip come from the subscriber. In contrast, on

weekend, most of them is on daytime, so the trip absolutely come from the customer who perhaps use the bikes for explore the city or travelling to visit place for enjoyment.

In [ ]:
```
plt.figure(figsize=(10,9))

plt.subplot(2, 1, 1)
top_5_start_station_long_trip = df_master.groupby(['start_station_name'])['dura
top_5_start_station_long_trip.iloc[:5].sort_values(ascending=True).plot(kind='
plt.title("Top 5 average trip duration of start station", size=14)
plt.ylabel("Station Name", size=14);

plt.subplot(2, 1, 2)
top_5_end_station_short_trip = df_master.groupby(['end_station_name'])['duratio
top_5_end_station_short_trip.iloc[:5].sort_values(ascending=True).plot(kind='ba
plt.title("Top 5 average trip duration end station", size=14);
```



Observation: The average trip duration of top 5 start/end stations seems not so much relevant to the scope of the analysis. These could be enhanced by categorizing the location to different types. To be better understanding, which type of location that the users are interested in the most.

Talk about some of the relationships you observed in this part of the investigation. How did the feature(s) of interest vary with other features in the dataset?

Differnt user type interestingly has different behaviors based on the time dimensions. And this affect to the number of trip & trip duration over the time of them. The customers ride longer trip than subscribers almost double the average trip time. Seasonal factors has the affect on the number of trip and the trip duration as well esepcially the month of holiday season and in cold weather like January & February. The weekend is the time of riding in which the trip duration is higher than weekdays about 4-5 minutes. Depend of the time frame of the day, people has the different trip duration but the variation is bigger as the fewer number of trip taken. Based on the weekend or weekdays, the number of trip also has different patterns. The busiest time in weekday is on starting and finishing office working time. Whereas, weekend riders take the most of the trip on daytime.

Did you observe any interesting relationships between the other features (not the main feature(s) of interest)?

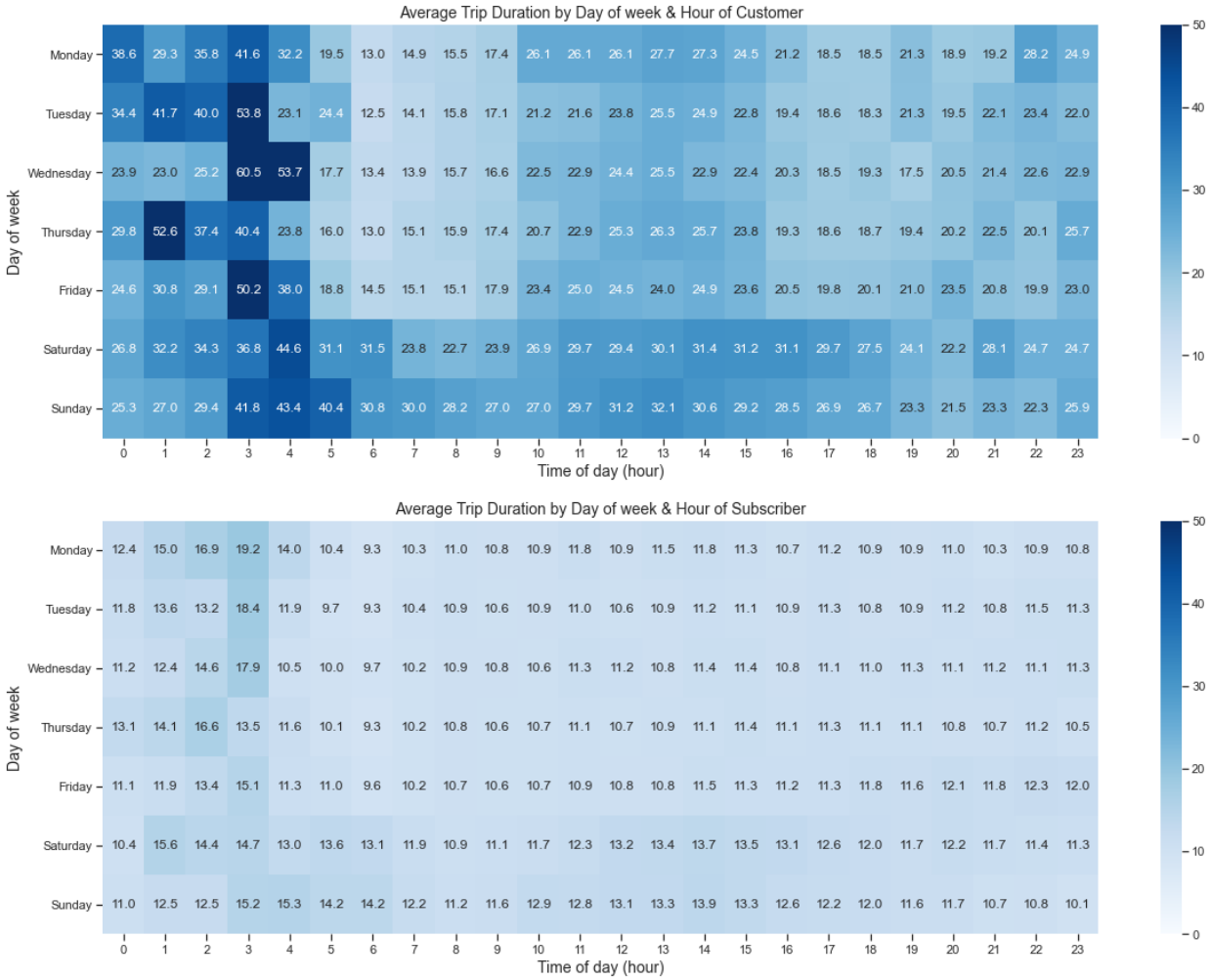
The top 5 strat/end average trip duration seem not so much relevant to the scope of this analysis. It requires more of information about location category to better understanding of the context to get further investigation on these features. Therefore, I will not include the multivariate exploration to these features.

Multivariate Exploration

In [ ]:
```
plt.figure(figsize=(20,15))

plt.subplot(2, 1, 1)
customers_df = df_master.query('user_type == "Customer"')
avg_duration_cust = customers_df.groupby(['day_of_week', 'time_of_day'])['dura
avg_duration_cust_df = avg_duration_cust.reset_index(name='avg_druation')
avg_duration_cust_pivot = avg_duration_cust_df.pivot(index='day_of_week', colu
sns.heatmap(avg_duration_cust_pivot, cmap="Blues", fmt=".1f", annot = True, ro
plt.title('Average Trip Duration by Day of week & Hour of Customer', fontsize=
plt.xlabel('Time of day (hour)', fontsize=14);
plt.ylabel('Day of week', fontsize=14);

plt.subplot(2, 1, 2)
subscriber_df = df_master.query('user_type == "Subscriber"')
avg_duration_subs = subscriber_df.groupby(['day_of_week', 'time_of_day'])['dur
avg_duration_subs_df = avg_duration_subs.reset_index(name='avg_druation')
avg_duration_subs_pivot = avg_duration_subs_df.pivot(index='day_of_week', colu
sns.heatmap(avg_duration_subs_pivot, cmap="Blues", fmt=".1f", annot = True, ro
plt.title('Average Trip Duration by Day of week & Hour of Subscriber', fontsiz
plt.xlabel('Time of day (hour)', fontsize=14);
plt.ylabel('Day of week', fontsize=14);
```



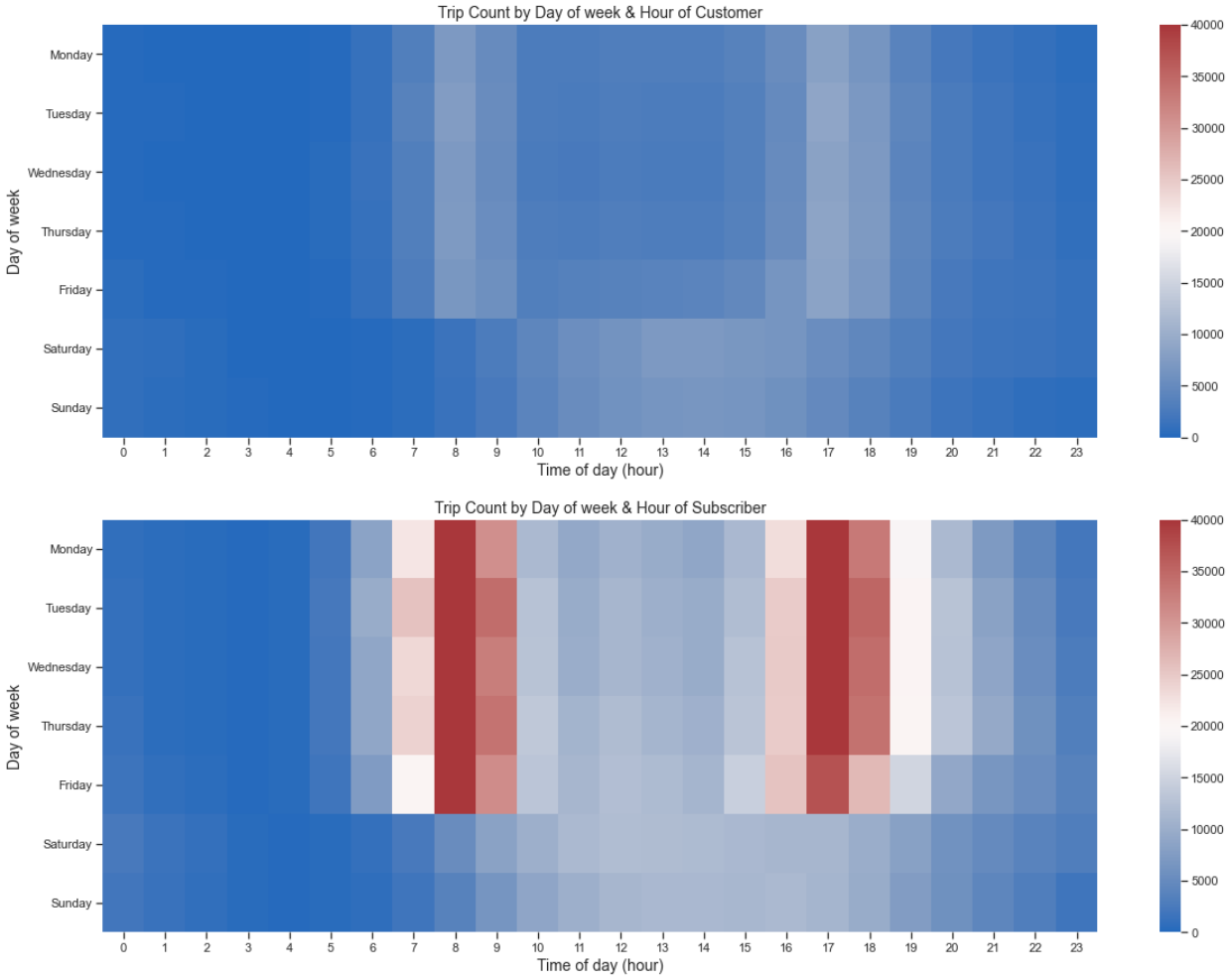
Observation: In overall, the customer rider has the average trip duration longer than the subscriber as stated in the color density of customer is darker for the whole-time frame of the day and 7 days of the week.

- Customer focus the trip on daytime of weekends. There is some significant average trip durations on early morning that could be the fewer number of trip taken during that time. The focus is on daytime from 10am till 6pm and on weekend to promote on weekend trip fun for the riders to getting to push the bike utilization on weekends.
- For Subscriber, almost the average trip duration is the same over the time of day and 7 days of the week. This could be different behavior from customer who could the tourists or travelers to get more time on rides rather than office employees who commute to work and finish the trip as soon as possible.

In [ ]:
```
plt.figure(figsize=(20,15))

plt.subplot(2, 1, 1)
customers_df = df_master.query('user_type == "Customer"')
trip_count_cust = customers_df.groupby(['day_of_week', 'time_of_day']).size()
trip_count_cust_df = trip_count_cust.reset_index(name='trip_count')
trip_count_cust_pivot = trip_count_cust_df.pivot(index='day_of_week', columns=
sns.heatmap(trip_count_cust_pivot, cmap="vlag", annot = False, robust=True, vm
plt.title('Trip Count by Day of week & Hour of Customer', fontsize=14);
plt.xlabel('Time of day (hour)', fontsize=14);
plt.ylabel('Day of week', fontsize=14);
```

```
plt.subplot(2, 1, 2)
subscriber_df = df_master.query('user_type == "Subscriber"')
trip_count_subs = subscriber_df.groupby(['day_of_week', 'time_of_day']).size()
trip_count_subs_df = trip_count_subs.reset_index(name='trip_count')
trip_count_subs_pivot = trip_count_subs_df.pivot(index='day_of_week', columns=
sns.heatmap(trip_count_subs_pivot, cmap="vlag", annot = False, robust=True, vm
plt.title('Trip Count by Day of week & Hour of Subscriber', fontsize=14);
plt.xlabel('Time of day (hour)', fontsize=14);
plt.ylabel('Day of week', fontsize=14);
```



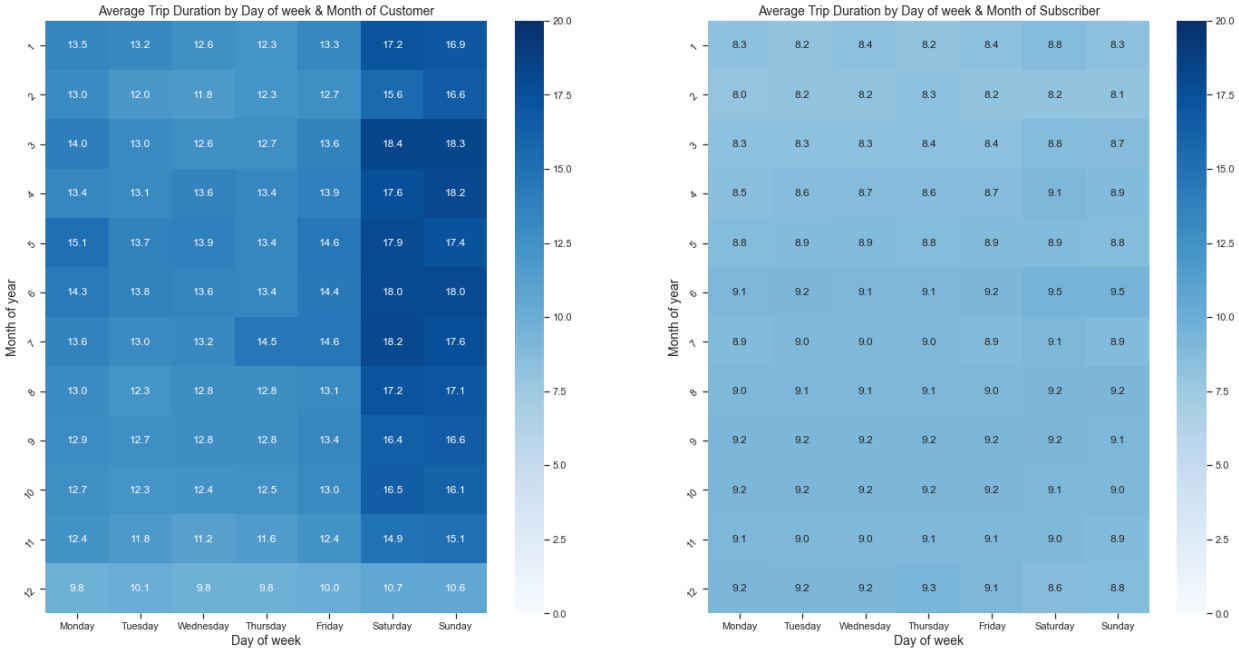
Observation: The heatmap clearly shows that customer use the bike on weekend (10am to 5pm) and weekday on 8am and it is getting denser on 5-6pm on weekdays. On the other hand, the subscriber uses the bike mostly on weekdays on 8am and 5pm. But in overall, the number of trips taken of subscriber is far higher than customer as the cooler color of customer.

In [ ]:
```
plt.figure(figsize=(24,12))

plt.subplot(1, 2, 1)
customers_df = df_master.query('user_type == "Customer"')
avg_duration_cust_2 = customers_df.groupby(['month_of_year', 'day_of_week'])['
avg_duration_cust_df_2 = avg_duration_cust_2.reset_index(name='avg_druation')
avg_duration_cust_pivot_2 = avg_duration_cust_df_2.pivot(index='month_of_year'
sns.heatmap(avg_duration_cust_pivot_2, fmt=".1f", cmap="Blues", annot = True,
plt.title('Average Trip Duration by Day of week & Month of Customer', fontsize=
plt.xlabel('Day of week', fontsize=14);
```

```
plt.ylabel('Month of year', fontsize=14)
plt.yticks(rotation = 45);

plt.subplot(1, 2, 2)
subscriber_df = df_master.query('user_type == "Subscriber"')
avg_duration_subs_2 = subscriber_df.groupby(['month_of_year', 'day_of_week'])[
avg_duration_subs_df_2 = avg_duration_subs_2.reset_index(name='avg_druation')
avg_duration_subs_pivot_2 = avg_duration_subs_df_2.pivot(index='month_of_year'
sns.heatmap(avg_duration_subs_pivot_2, fmt=".1f", cmap="Blues", annot = True,
plt.title('Average Trip Duration by Day of week & Month of Subscriber', fontsi
plt.xlabel('Day of week', fontsize=14);
plt.ylabel('Month of year', fontsize=14)
plt.yticks(rotation = 45);
```



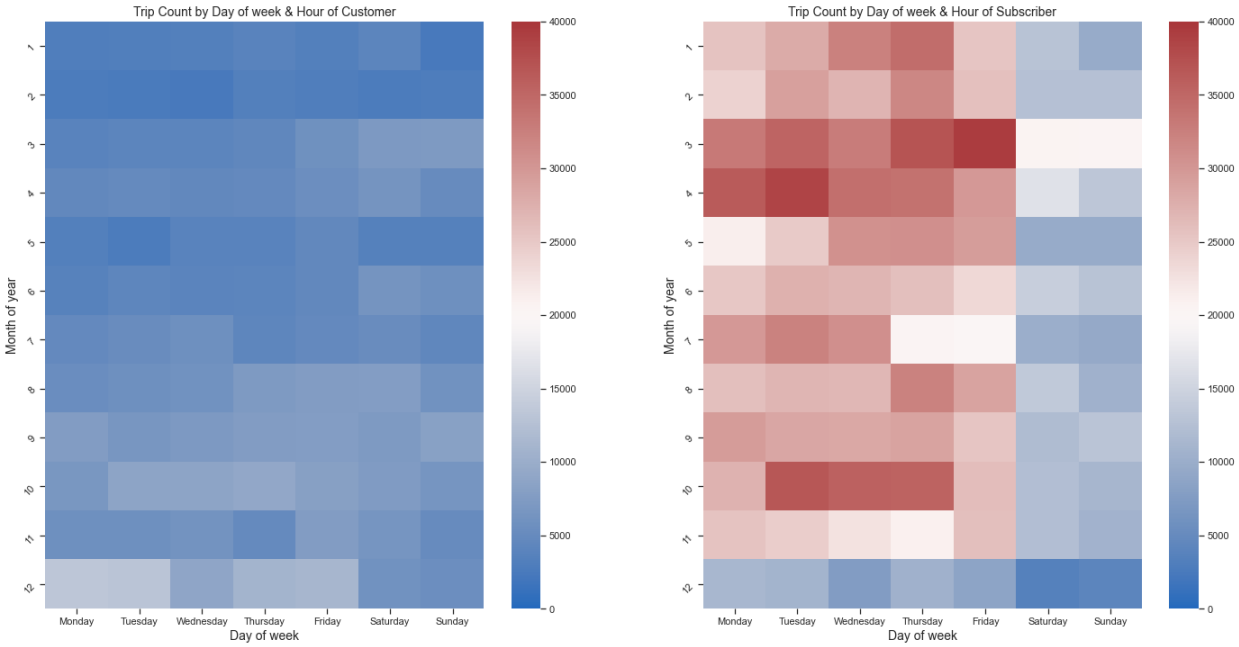
Observation:

- The seasonal factor has the effect on customer biker's behavior on summertime (May, June & July) where the average trip on day of week of those months are denser from the rest of the month. This could be the season where people have the summer holiday and traveler will take the riders on those days.
- The stable average trip duration of subscriber shows clearly on the heatmap over the weeks and months. They are expected to use the bike only for going to work.
- Average trip duration drops in December for both type of user.

In [ ]:
```
plt.figure(figsize=(24,12))

plt.subplot(1, 2, 1)
customers_df = df_master.query('user_type == "Customer"')
avg_duration_cust_2 = customers_df.groupby(['month_of_year', 'day_of_week']).s
avg_duration_cust_df_2 = avg_duration_cust_2.reset_index(name='avg_druation')
avg_duration_cust_pivot_2 = avg_duration_cust_df_2.pivot(index='month_of_year'
sns.heatmap(avg_duration_cust_pivot_2, cmap="vlag", annot = False, robust=True
plt.title('Trip Count by Day of week & Hour of Customer', fontsize=14);
plt.xlabel('Day of week', fontsize=14);
plt.ylabel('Month of year', fontsize=14)
plt.yticks(rotation = 45);
```

```
plt.subplot(1, 2, 2)
subscriber_df = df_master.query('user_type == "Subscriber"')
avg_duration_subs_2 = subscriber_df.groupby(['month_of_year', 'day_of_week'])[
avg_duration_subs_df_2 = avg_duration_subs_2.reset_index(name='avg_druation')
avg_duration_subs_pivot_2 = avg_duration_subs_df_2.pivot(index='month_of_year',
sns.heatmap(avg_duration_subs_pivot_2, cmap="vlag", annot = False, robust=True,
plt.title('Trip Count by Day of week & Hour of Subscriber', fontsize=14);
plt.xlabel('Day of week', fontsize=14);
plt.ylabel('Month of year', fontsize=14)
plt.yticks(rotation = 45);
```



Observation:

- The heatmap shows most of the trip taken over the year of subscriber is on Monday to Friday and from January to November with the peaks Fridays of March, Tuesdays of April, and Tuesdays of October and the number significant drops in December. In contrast, the customer is getting hot in December from Monday till Friday. That is a bit surprising insight that we can customize the promoting and bike utilization based on the pattern usage of user type on different time dimensions.

Talk about some of the relationships you observed in this part of the investigation. Were there features that strengthened each other in terms of looking at your feature(s) of interest?

- Cusomter riders take longer trip in average than subscriber on every time dimensions. The seasonal factors indicates clearly on the heatmap over months on trip count and average trip duration of cutomers.

Were there any interesting or surprising interactions between features?

The only month of the year is December, the cusomter will take the trip more than the subscriber.

Conclusions

- 82.5% of trip is conducted on weekday rather than weekend. In addition, there are 80.38% of the trip is taken by subscriber than customer.
- Customer trip duration distribution is typically from 10 to 20 minutes that is longer than subscribers whose ride mostly is from 8 minutes to 10 minutes. And most of the customer is taking the longer trip which is longer than 50 minutes.
- Although the number of trips conducted by customer is smaller than subscriber, their average trip duration (22 minutes) is double from subscriber (11 minutes).
- Average trip duration in 2019 is 13.41 min. The number of trips on Saturday & Sunday is lower compared to other days in weekdays. However, there is a big difference in the trip duration on weekends. People usually ride about 4-5 min longer than weekdays.
- Overall, the subscriber contributes the greatest number of the trip over the months of 2019. However, in December, trip count's customer is higher than subscriber. This could be because of the holiday season so our regular bikers are not going to commute to work (perhaps they take annual leaves) rather than the customer could be tourists or travelers from outside of the city.
- The month of using for subscriber is in March and April. After that the number of trips is dropping as coming to summertime and come back on springtime (September & October). And it significant dops on holiday season when most of office workers is taking time off from work and not commuting to work. On the other hand, customer starts using more bikes from September till end of the year.
- On weekdays trips are mostly occurs on the time frame of commuting to work of office workers. As a result, most of the trip come from the subscriber. In contrast, on weekend, most of them is on daytime, so the trip absolutely come from the customer who perhaps use the bikes for explore the city or travelling to visit place for enjoyment.
- Customer rider has the average trip duration longer than the subscriber for the whole-time frame of the day and 7 days of the week.
- Customer focus their trip on the daytime of weekends. There is a significant average trip duration on early morning that could be the fewer number of trips taken during that time. The focus is on daytime from 10am till 6pm and on weekend to promote on weekend trip fun for the riders to getting to push the bike utilization on weekends.
- For Subscriber, almost the average trip duration is the same over the time of day and 7 days of the week. This could be different behavior from customer who could be tourists or travelers to spend more time on rides rather than office employees who commute to work and try to finish the trip as soon as possible.
- The seasonal factor has the effect on customer biker’s behavior on summertime (May, June & July) where the average trip on day of week of those months are denser from

the rest of the month. This could be the season where people have the summer holiday and traveler will take the riders on those days.

- The stable average trip duration of subscriber shows clearly on the heatmap over the weeks and months. They are expected to use the bike only for going to work.
- Average trip duration drops in December for both type of user.
- Most of the trip taken over the year of subscriber is on Monday to Friday and from January to November with the peaks Fridays of March, Tuesdays of April, and Tuesdays of October and the number significant drops in December. In contrast, the customer is getting hot in December from Monday till Friday. That is a bit surprising insight that we can customize the promoting and bike utilization based on the pattern usage of user type on different time dimensions.